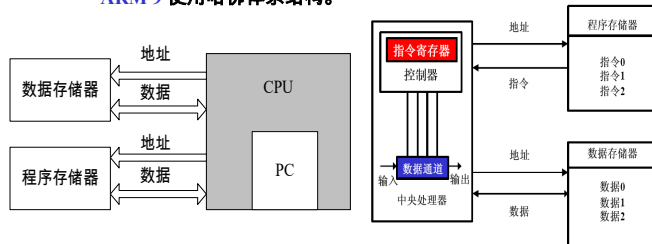




- ARM 微处理器概述
- ARM 微处理器结构
- ARM 微处理器的工作状态
- ARM 体系结构的存储器格式
- 处理器模式
- 寄存器组织
- 异常

哈佛体系结构

- 哈佛机：为数据和程序提供了各自独立的存储器。
- 程序计数器只指向程序存储器而不指向数据存储器，这样做的后果是很难在哈佛机上编写出一个自修改的程序。
- 独立的程序存储器和数据存储器为数字信号处理提供了较高的性能。
- ARM 7 使用冯·诺依曼体系结构。
- ARM 9 使用哈佛体系结构。



2.1.1 ARM 微处理器的特点

- 采用 RISC 架构的 ARM 微处理器一般具有如下特点：
 - 1. 体积小、低功耗、低成本、高性能；
 - 2. 支持 Thumb (16 位) / ARM (32 位) 双指令集，能很好的兼容 8 位 / 16 位器件；
 - 3. 大量使用寄存器，指令执行速度更快；
 - 4. 大多数数据操作都在寄存器中完成；
 - 5. 寻址方式灵活简单，执行效率高；
 - 6. 指令长度固定。

- 两种典型的结构：
 - 冯·诺依曼结构
 - 哈佛体系结构



ARM

- ARM 是 Advanced RISC Machines 的缩写，它是一家微处理器行业的知名企业，该企业设计了大量高性能、廉价、耗能低的 RISC (精简指令集) 处理器。1985 年第一个 ARM 原型在英国剑桥诞生。

- ARM 公司的特点是只设计芯片，而不会提供 ARM 技术知识产权 (IP) 核，将技术授权给世界上许多著名的半导体、软件和 OEM 厂商，并提供服务。



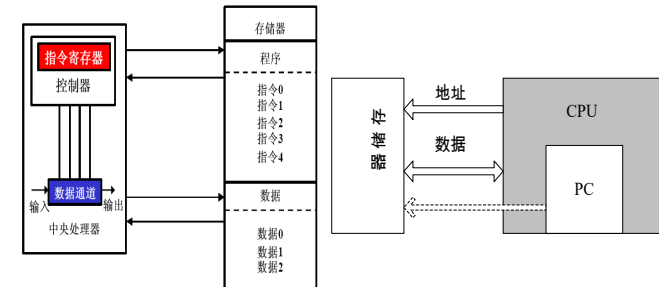
- ARM，可以认为是：
 - 一个公司的名字
 - 一类微处理器的通称
 - 一种技术的名字

ARM 体系结构版本

- ARM 体系结构从最初开发到现在有了很大的改进，并仍在完善和发展。
- 为了清楚地表达每个 ARM 应用实例所使用的指令集，ARM 公司定义了主要的 ARM 指令集体系结构版本，以版本号 V1..... 表示。
- ARM 处理器系列中的各种处理器，虽然在实现技术、应用场合和性能方面都不尽相同，但只要支持相同的 ARM 体系版本，基于它们的应用软件将是兼容的。

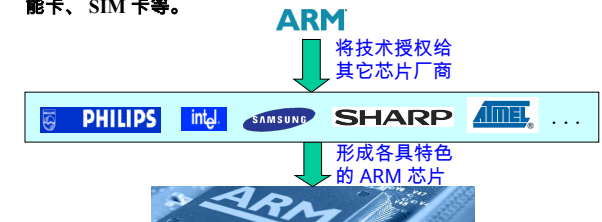


- 冯·诺依曼机：将数据和指令都存储在存储器中的计算机。
- 计算系统由一个中央处理单元 (CPU) 和一个存储器组成。存储器拥有数据和指令，并且可以根据所给的地址对它进行读或写。



ARM 处理器的应用

- 2011 年，ARM 的客户报告了 79 亿 ARM 处理器出货量，占有 95 % 的智能手机、90 % 的硬盘驱动器、40 % 的数字电视和机顶盒、15 % 的单片机、和 20 % 的移动计算机。在 2012 年，微软与 ARM 科技生产了新的 Surface 平板电脑，AMD 宣布它将于 2014 年开始生产基于 ARM 核心的 64 位服务器芯片。
- “掌上计算”相关的所有领域皆为其所主宰。
- 主要应用：消费类电子、无线、图像应用开放平台、存储、自动化、智能卡、SIM 卡等。



ARM 版本：V1 版架构

- 应用：原型机 ARM1
- 寻址地址：26 位
- 产品化：没有用于商业产品。

基本性能：

- 数据处理：基本的数据处理指令 (无乘法)。
- 存储：基于字节、半字和字的 Load/Store 指令。
- 转移指令：包括子程序调用及链接指令。
- 中断：供操作系统使用的软件中断指令 SWI。
- 寻址空间：64MB (2²⁶) 字节。

ARM 版本：V2 版架构

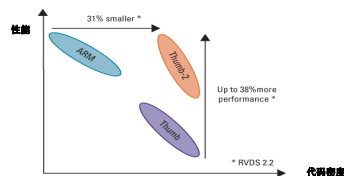
- V2 是对 V1 的扩展。
- 应用：ARM2\ARM3 使用的是 V2a 架构。版本 2a 是版本 2 的变种，ARM3 芯片采用了版本 2a，是第一片采用片上 Cache 的 ARM 处理器。
- 现在已经废弃不再使用。
- 对 V1 功能改进：
 - 数据处理：乘法和乘加指令。
 - 协处理：支持协处理器操作指令。
 - 中断：快速中断模式。
 - 寻址：SWP/SWPB 的最基本存储器与寄存器交换指令。

ARM 版本：V5 版架构

- V5 版架构是在 V4 版基础上增加了一些新的指令，ARM10 和 Xscale 都采用该版架构。
- 新增命令：
 - 转移：带有链接和交换的转移 BLX 指令。
 - 数据处理：计数前导零 CLZ 指令。
 - 中断：BRK 中断指令。
 - 数字信号与协处理器：增加了数字信号处理指令（V5TE 版），为协处理器增加更多可选择的指令。
 - 双指令集：改进了 ARM/Thumb 状态之间的切换效率。
- E--- 增强型 DSP 指令集，包括全部算法操作和 16 位乘法操作。
- J--- 支持新的 JAVA，提供字节代码执行的硬件和优化软件加速功能。

Thumb-2 指令集

- Thumb 指令集的目标是实现更高的代码密度，是 ARM 指令集的子集。在指令的执行阶段，16 位的指令被重新解码，完成对等的 32 位指令所实现的功能。使用 Thumb 指令在代码密度方面改善大约 30%，但这种改进是以代码的效率为代价的。
- Thumb-2 指令集在现有的 Thumb 指令的基础上做了扩充：增加了一些 16 位 Thumb 指令改进程序的执行流程，增加了一些 32 位 Thumb 指令实现一些 ARM 指令的专有功能，解决了 Thumb 指令集不能访问协处理器、特权指令和特殊功能指令的局限。
- Thumb-2 指令集不需要在 ARM/Thumb 状态之间反复切换，代码密度和性能得到的显著的提高。



ARM 版本：V3 版架构

- V3 是对 V2 的改进。V3 版架构（目前已废弃）对 ARM 体系结构作了较大的改动。
- 应用：ARM6。1990 年 ARM 公司设计的第一个微处理器采用的是 V3 的 ARM6。
- 增加功能：
 - 寻址空间：增至 32 位（4GB）。
 - 程序状态信息：
 - 当前程序状态信息从原来的 R15 寄存器移到一个新的寄存器中，即当前程序状态寄存器 CPSR。
 - 增加了程序状态保存寄存器 SPSR，保存程序异常中断时的程序状态，以便于对异常（Exception）的处理。
 - 增加了 MRS/MSR 指令，以访问新增的 CPSR/SPSR 寄存器。
- 中断：
 - 增加了中止（Abort）和未定义两种处理器模式。
 - 增加了从异常处理返回的指令功能。

ARM 版本：V6 版架构

- V6 版架构是 2001 年发布的，首先在 2002 年春季发布的 ARM11 处理器中使用。
- 在降低耗电量地同时，还强化了图形处理性能。通过追加有效进行多媒体处理的 SIMD(Single Instruction, Multiple Data，单指令多数据) 功能，将语音及图像的处理功能提高到了原型机的 4 倍。
- 增加功能：
 - THUMB™：35% 代码压缩。
 - DSP 扩充：高性能定点 DSP 功能。
 - Jazelle™：Java 性能优化，可提高 8 倍。
 - Media 扩充：音/视频性能优化，可提高 4 倍。

ARM 版本：V8 版架构

- ARM v8 架构在 2011 年 11 月公司发布，具备 64 位计算能力。
- 64 位通用寄存器、SP（堆栈指针）和 PC（程序计数器）
- 64 位数据处理和扩展的虚拟寻址
- 两种主要执行状态：
 - AArch64 - 64 位执行状态，包括该状态的异常模型、内存模型、程序员模型和指令集支持
 - AArch32 - 32 位执行状态，包括该状态的异常模型、内存模型、程序员模型和指令集支持
- 这些执行状态支持三个主要指令集：
 - A32（或 ARM）：32 位固定长度指令集，通过不同架构变体增强部分 32 位架构执行环境现在称为 AArch32。
 - T32 (Thumb) 是以 16 位固定长度指令集的形式引入的，随后在引入 Thumb-2 技术时增强为 16 位和 32 位混合长度指令集。部分 32 位架构执行环境现在称为 AArch32。
 - A64：提供与 ARM 和 Thumb 指令集类似功能的 32 位固定长度指令集。随 ARMv8 一起引入，它是一种 AArch64 指令集。

ARM 版本：V4 版架构

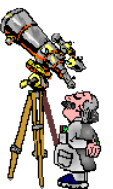
- V4 在 V3 上进一步扩充。
- V4 版架构是目前应用最广的 ARM 体系结构，ARM7、ARM8、ARM9 和 StrongARM 都采用该架构。
- V4 不再强制要求与 26 位地址空间兼容，而且还明确了哪些指令会引起未定义指令异常。
- 指令集中增加了以下功能：
 - 数据处理：符号化和非符号化半字及符号化字节的存/取指令。
 - 双指令集：增加了 T 变种，处理器可工作在 Thumb 状态，增加了 16 位 Thumb 指令集。
 - 中断：
 - 完善了软件中断 SWI 指令的功能。
 - 处理器系统模式进入特权方式时使用用户寄存器操作。
 - 把一些未使用的指令空间捕获为未定义指令。

ARM 版本：V7 版架构

- ARMv7 架构是在 ARMv6 架构的基础上诞生的，2010 年推出的指令集架构。
- 该架构采用了 Thumb-2 技术，它是在 ARM 的 Thumb 代码压缩技术的基础上发展起来的，并且保持了对现存 ARM 解决方案的完整的代码兼容性。
- Thumb-2 技术比纯 32 位代码少使用 31% 的内存，减小了系统开销，同时能够提供比已有的基于 Thumb 技术的解决方案高出 38% 的性能。
- ARMv7 架构还采用了 NEON 技术，将 DSP 和媒体处理能力提高了近 4 倍。并支持改良的浮点运算，满足下一代 3D 图形、游戏物理应用以及传统嵌入式控制应用的需求。

2.1.2 ARM 微处理器系列

- 目前市面上常见的处理器主要包括 ARM7、ARM9、ARM9E、ARM10E、SecurCore、Cortex 等几个系列，以及其他厂商基于 ARM 体系结构的处理器。除了具有 ARM 体系结构的共同特点以外，每一个系列的 ARM 微处理器都有各自的特点和应用领域。
- ARM7、ARM9、ARM9E 和 ARM10 为 4 个通用处理器系列。每一个系列提供一套相对独特的性能来满足不同应用领域的需求。
- SecurCore 系列专门为安全要求较高的应用而设计。
- ARM7 系列
- ARM9 系列
- ARM9E 系列
- ARM10E 系列
- SecurCore 系列
- Inter 的 StrongARM
- Inter 的 Xscale
- Cortex



ARM7 微处理器系列

- 低功耗的 **32 位 RISC** 处理器，**冯·诺依曼结构**。
 - 具有嵌入式 ICE - RT 逻辑，调试开发方便。
 - 3 级流水线结构**。
 - 代码密度高，兼容 16 位的 Thumb 指令集。
 - 对操作系统的支持广泛，包括 Windows CE、Linux、Palm OS 等。
 - 指令系统与 ARM9 系列、ARM9E 系列和 ARM10E 系列兼容，便于用户的产品升级换代。
 - 主频最高可达 130MIPS。
- **主要应用领域：**工业控制、Internet 设备、网络和调制解调器设备、移动电话等多种多媒体和嵌入式应用。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 19

ARM10E 微处理器系列

- 与同等的 ARM9 比较，在同样的时钟频率下，性能提高了近 50%，功耗极低。
 - 支持 DSP 指令集。
 - 6 级整数流水线**，指令执行效率更高。
 - 支持 32 位 ARM 指令集和 16 位 Thumb 指令集。
 - 支持 VFP10 浮点处理协处理器。
 - 全性能的 **MMU**，支持 Windows CE、Linux、Palm OS 等多种主流嵌入式操作系统。
 - 支持数据 Cache 和指令 Cache**。
 - 主频最高可达 400MIPS。
 - 内嵌并行读/写操作部件。
- **主要应用：**下一代无线设备、数字消费品、成像设备、工业控制、通信和信息系统等领域。
- **3 种类型：**ARM1020E、ARM1022E 和 ARM1026EJ-S。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 22

Xscale 处理器

- 基于 **ARMv5TE** 体系结构的解决方案。
 - 是一款全性能、高性价比、低功耗的处理器。
 - 支持 16 位的 Thumb 指令和 DSP 指令集。
- **主要应用：**数字移动电话、个人数字助理和网络产品。
- Xscale 处理器是 Intel 目前主要推广的一款 ARM 微处理器。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 25

ARM9 微处理器系列

- ARM9 系列微处理器在高性能和低功耗特性方面提供最佳的性能。
 - 5 级整数流水线**。
 - 哈佛体系结构**。
 - 支持 32 位 ARM 指令集和 16 位 Thumb 指令集。
 - 全性能的 MMU，支持 Windows CE、Linux、Palm OS 等多种主流嵌入式操作系统。
 - 支持数据 Cache 和指令 Cache**，具有更高的指令和数据处理能力。
- **主要应用：**无线设备、仪器仪表、安全系统、机顶盒、高端打印机、数码相机和数码摄像机。
- **3 种类型：**ARM920T、ARM922T 和 ARM940T。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 20

SecurCore 微处理器系列

- 专为 **安全** 需要而设计，提供了完善的 32 位 RISC 技术的安全解决方案。
 - 灵活的保护单元，以确保操作系统和应用数据的安全。
 - 采用软内核技术，防止外部对其进行扫描探测。
 - 可集成用户自己的安全特性和其他协处理器。
- **主要应用：**对安全性要求较高的应用产品及应用系统，如电子商务、电子政务、电子银行业务、网络和认证系统等。
- **4 种类型：**SecurCore SC100、SecurCore SC110、SecurCore SC200 和 SecurCore SC210。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 23

ARM Cortex 系列：V7 版架构

- Cortex 系列属于 ARMv7 架构。
 - ARMv7 架构中，内核架构首次从单一款式变成 3 种款式：
 - A 系列：Application Processor**，面向尖端的基于虚拟内存的操作系统和用户应用——越来越接近电脑。
 - R 系列：Real Time Control**，针对实时系统——既快又实时。
 - M 系列：Micro Controller**，对微控制器——单片机风格。
- 由于应用领域不同，基于 v7 架构的 Cortex 处理器系列所采用的技术也不相同：
- 基于 v7 A 的称为 Cortex-A 系列。
 - 基于 v7 R 的称为 Cortex-R 系列。
 - 基于 v7 M 的称为 Cortex-M 系列。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 26

ARM9E 微处理器系列

- 单一处理器内核提供微控制器、DSP、Java 应用系统的解决方案。
- 支持 DSP 指令集。
- 5 级整数流水线**，指令执行效率更高。
- 支持 32 位 ARM 指令集和 16 位 Thumb 指令集。
- 支持 VFP9 浮点处理协处理器。
- 全性能的 **MMU**，支持 Windows CE、Linux、Palm OS 等多种主流嵌入式操作系统。
- MPU 支持实时操作系统。
- 支持数据 Cache 和指令 Cache**。
- 主频最高可达 300MIPS。
- 主要应用：**下一代无线设备、数字消费品、成像设备、工业控制、存储设备和网络设备等领域。
- 3 种类型：**ARM926EJ-S、ARM946E-S 和 ARM966E-S。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 21

StrongARM 微处理器系列

- Inter StrongARM SA-1100 系列微处理器是采用 ARM 体系结构高度集成的 32 位 RISC 微处理器。
 - 融合了 Intel 公司的设计和处理技术，以及 ARM 体系结构的电源效率。
 - 采用在 **软件上兼容 ARM V4**，同时兼具 Intel 技术优点的体系结构。
- **主要应用：**便携式通信产品，消费类电子产品，掌上电脑系列产品。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 24

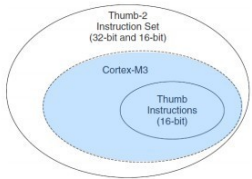
ARM Cortex 系列

- Cortex-A 系列：**需要运行复杂应用程序的“应用处理器”，支持大型嵌入式操作系统，比如 Symbian（诺基亚智能手机用）、Linux，以及微软的 Windows CE 和智能手机操作系统 Windows Mobile。这些应用需要高的处理性能，需要硬件 MMU 实现的完整而强大的虚拟内存机制，还会配有 Java 支持，有时还要求一个安全程序执行环境（用于电子商务）。典型的产品包括高端手机和手持仪器，电子钱包以及金融事务处理机。如：**Cortex-A8**（应用处理器）
- Cortex-R 系列：**硬实时且高性能的处理器。目的是高端实时市场。像高档轿车的组件、大型发电机控制器、机器人臂控制器等，它们使用的处理器不但要很好很强大，还要极其可靠，对事件的反应也要极其敏捷。如：**Cortex-R4**（实时处理器）。
- Cortex-M 系列：**为单片机的应用而量身定制。在这些应用中，尤其是对于实时控制系统，低成本、低功耗、极速中断反应以及高处理效率，都是至关重要的。其中 **Cortex-M3** 就是按款式 M 设计的。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 27

Cortex-M3 处理器

- Cortex-M3 处理器是**基于 ARMv7-M 架构**的处理器，3 级流水，分支预测，采用了纯 **Thumb2** 指令的执行方式，具有极高的运算能力和中断响应能力。
- 主要应用**：汽车车身系统、工业控制系统、无线网络等对功耗和成本敏感的嵌入式应用领域。
- 目前最便宜的基于 Cortex-M3 内核的 ARM 单片机售价为 1 美元。



ARM 系列流水线比较

ARM7 3 级	预取 (Fetch)	译码 (Decode)	执行 (Execute)					
ARM9 5 级	预取 (Fetch)	译码 (Decode)	执行 (Execute)	访存 (Memory)	写入 (Write)			
ARM10 6 级	预取 (Fetch)	发送 (Issue)	译码 (Decode)	执行 (Execute)	访存 (Memory)	写入 (Write)		
ARM11 8 级	预取 (Fetch)	预取 (Fetch)	发送 (Issue)	译码 (Decode)	转换 (Snny)	执行 (Execute)	访存 (Memory)	写入 (Write)

ARM 系列参数比较 2

ARM9E	ARM926EJ-S ：Jazelle 技术，有 MMU，可配置的数据和指令 Cache、TCM 接口； ARM946E-S ：可配置的数据和指令 Cache 及 TCM； ARM966E-S ：针对要求高性能和低功耗的可预测的指令执行时间的硬实时应用设计 ARM968E-S ：最小、功耗最小的 ARM9E 系列处理器，针对嵌入式实时应用设计；	●ARM9E 是针对微控制器、DSP 和 Java 的单处理器解决方案； ●ARM Jazelle 技术提供 8 倍的 Java 加速性能 ● ARM926EJ-S ： ●5-级整数流水线； ●在 0.13μm 工艺下最高性能可达到 300MIPS (Dhrystone 2.1 测试标准)； ●可选择的 向量浮点单元 VFP9 协处理器指令优秀海浮点性能，对于 3D 图形加速和实时控制可达到 215MFLOPS。 ●高性能的 AHB 总线，带 MMU ●可在 0.18μm、0.15μm、0.13μm 工艺的硅芯片上实现。
ARM10E	ARM1020E ：带 DSP 指令集，在片调试功能，独立的 32KB 数据和指令 Cache，MMU 支持； ARM1022E ：与 ARM1020E 相同，只是独立的数据和指令 Cache 变为 16KB； ARM1026EJ-S ：同时具有 MPU 和 MMU，可综合版本	●带分支预测的 6 级整数流水线； ●在 0.13μm 工艺下最高性能可达到 430MIPS (Dhrystone 2.1 测试标准)； ●对于 3D 图形运算和实时控制采用 VFP 协处理器，浮点运算性能最高可达 650MFLOPS； ●双 64 位 AMBA 总线接口和 64 位内部总线接口； ●优化的缓存结构提高了处理器访问低速存储器的性能； ●可在 0.18μm、0.15μm、0.13μm 工艺的硅芯片上实现

Cortex-R4 处理器

- Cortex-R4 处理器是首款基于 ARMv7 架构的高级嵌入式处理器。
- Cortex-R4 提供的性能、实时响应性大大高于同类中的其他处理器，它提供的功能也远远多于同类中的其他处理器。
- 主要目标**：产量巨大的高级嵌入式应用系统，如硬盘，喷墨式打印机，以及汽车安全系统等等。
- Cortex-R4 主要功能**：
 - 快速**：指令预取和分支预测，**8 级双指令流水线**，Thumb-2 / ARM 指令，硬件除法、SIMD、DSP、哈佛结构。
 - 确定性**：快速中断响应，矢量中断控制器端口，低延迟中断模式，除了用于存储重要的代码和数据（如中断服务例程无需等待 cache 逐出再从主内存读取就可以立即执行），紧密耦合存储系统也提供第二个 1 级存储。
 - 可靠性**：构建于内核中的错误处理，内存保护单元，1 级内存的 ECC 和奇偶校验保护，双核锁步配置。
 - 经济实惠**：成本低。

ARM 系列性能比较

项目	ARM7	ARM9	ARM10	ARM11
流水线	3	5	6	8
典型频率 (MHz)	80	150	260	335
功耗 (mW/MHz)	0.06	0.19 (+cache)	0.5 (+cache)	0.4 (+cache)
性能 MIPS/MHz	0.97	1.1	1.3	1.2
架构	冯·诺伊曼	哈佛	哈佛	哈佛

ARM 系列参数比较 3

ARM11	ARM11 MPCore ：可综合的多处理器核，1 至 4 个处理器可配置； ARM1136J(F)-S ：可配置的数据和指令 Cache，可提供 1.9 位的 MPEG4 编码加速功能； ARM1156T2(F)-S ：带集成浮点协处理器，带内存保护单元 MPU； ARM1176JZ(F)-S ：带针对 CPU 和系统安全架构扩展的 TrustZone 技术。	●增强的 Thumb、Jazelle、DSP 扩展支持； ●带片上和系统安全 TrustZone 技术支持； ●在 0.13μm 工艺下最高可达到 550MHz； ●MPCore 在 0.13μm 工艺下最高性能可达到 740MIPS (Dhrystone 2.1 测试标准)； ●支持多媒体指令 SIMD； ●采用三种电源模式：全速 / 待命 / 休眠 ●集成 DMA 的 TCM ●低功耗、高性能。
Secur Core	SC100 ：第一个 32 位安全处理器； SC110 ：在 SC100 上增加密钥协处理器； SC200 ：带 Jazelle 技术的高级安全处理器； SC210 ：在 SC200 上增加密钥协处理器	●SecurCore 是专门为智能卡、安全 IC 提供的 32 位安全处理器，为电子商务、银行、网络、移动多媒体、公共交通提供安全解决方案； ●体积小、功耗低，代码压缩密度高； ●为快速增长的 Java 卡平台提供 Java 加速功能；

Cortex-A8 处理器

- ARM Cortex-A8 处理器是第一款**基于 ARMv7 架构的应用处理器**，并且是 ARM 开发的性能最高、最具功率效率的处理器。ARM Cortex-A8 处理器有 **13 级流水线**，带有先进的**动态分支预测**。
- Cortex-A8 处理器的速率可以在 600MHz 到超过 1GHz 的范围内调节，能够满足那些需要工作在 300mW 以下的功耗优化的移动设备的要求。
- Cortex-A8 处理器是 ARM 的第一款**超标量处理器**，具有提高代码密度和性能的技术，已在工业控制、医疗电子、节能环保、智能交通、能源节能、电力系统、通讯系统、纺织行业、数控行业、汽车电子、工业触摸屏控制系统、机器人视觉、媒体处理无线应用、数字家电、车载设备、通信设备、网络终端等环境恶劣场合广泛应用。

ARM 系列参数比较 1

ARM 系列	微处理器核	特点
ARM7	ARM7TDMI ：整数处理核 ARM7TDMI 处理器的可综合版本； ARM720T ：带 MMU 的处理器核心，支持操作系统； ARM7EJ-S ：带有 DSP 和 Jazelle™ 技术，能够实现 Java 加速功能	●冯·诺伊曼体系结构； ●ARM7TDMI 是目前应用最广的微处理器核 ●ARM720T 带有 MMU 和 8KB 的指令数据混合 cache； ●ARM7EJ-S 执行 ARMv5TEJ 指令，5 级流水线，提供 Java 加速指令，没有存储器保护。
ARM9	ARM920T ：带有独立的 16KB 数据和指令 Cache； ARM922T ：带有独立的 8 位 KB 数据和指令 Cache； ARM940T ：包括更小数据和指令 Cache 和一个 MPU	●基于 ARM9TDMI，带 16 位的 Thumb 指令集，增强代码密度最多到 35%； ●在 0.13μm 工艺下最高性能可达到 300MIPS (Dhrystone 2.1 测试标准)； ●集成了数据和指令 Cache； ●32 位 AMBA 总线接口的 MMU 支持； ●可在 0.18μm、0.15μm 和 0.13μm 工艺的硅芯片上实现。

ARM 系列参数比较 4

Cortex	Cortex-A ：面向应用的微处理器，针对复杂操作系统和应用程序设计； Cortex-R ：针对实时系统的嵌入式处理器； Cortex-M ：针对成本敏感应用优化的深度嵌入式处理器；	●2004 年发布，提供增强的媒体和数字处理能力，增加了系统性能； ●支持 ARM、Thumb、Thumb-2 指令集； ●Thumb-2 指令集提供了更高的代码存储密度，进一步降低成本；
Intel 系列	StrongARM ：ARMv4 体系 XScale ：ARMv5TE 体系，增加 MMX 指令	●StrongARM 主要应用于手持设备和 PDA，5 级流水线，具有独立的数据和指令 Cache，不支持 Thumb 指令集，目前已停产； ●XScale 是目前 Intel 公司主推的高性能嵌入式处理器，分通用处理器、网络处理器和 I/O 处理器三类。其中通用处理器有 PXA25x、PXA26x、PXA27x 三个系列，被广泛应用于智能手机、PDA 领域。

ARM 发展历程 - ARM 公司

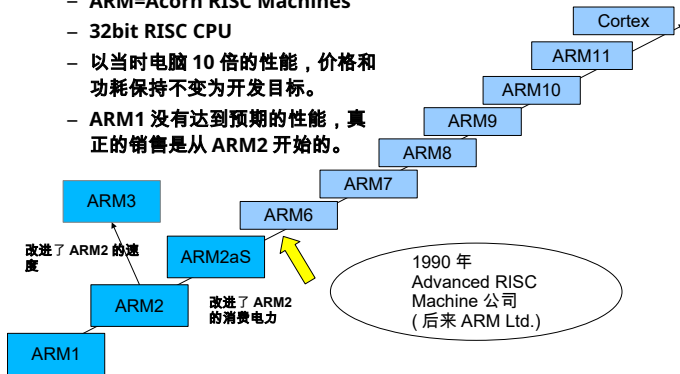
- 英文名称：Advanced RISC Machines
- 国家：英国（欧洲）
- 行业：电子 半导体 微处理器 智能手机
- 总部：英国剑桥
- 成立时间：1990 年
- 公司网址：www.arm.com
- 员工：2200
- CEO：西蒙·希加斯（Simon Segars）
- 分支机构：比利时、法国、印度、瑞典、美国、中国上海等 30 多家。
- 竞争对手：英特尔
- 市场份额：
 - 智能手机处理器 95%
 - 上网本处理器 30%
 - 平板电脑处理器 80%
 - 硬盘控制处理器 90%



第三任 CEO 西蒙·希加斯（2013 年 7 月上任）

ARM 发展历程 3

- ARM1 的开发
 - ARM=Acorn RISC Machines
 - 32bit RISC CPU
 - 以当时电脑 10 倍的性能，价格和功耗保持不变为开发目标。
 - ARM1 没有达到预期的性能，真正的销售是从 ARM2 开始的。



“ARM 处理器系列”命名规则 3

- 命名格式：ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {-S}
- 在 ARM 处理器内核中有多个功能模块可供生产厂商选择。这些模块分别用 T、D、M、I、E、J、F、S 来表示，从处理器的内核版本上可以区分出来。
- T：Thumb，Thumb16 位译码器，说明该内核可从 16 位 Thumb 指令集扩充到 32 位 ARM 指令集。V4T 以上版本。
- D：Debug，JTAG 调试器，说明该内核中内置了用于调试的结构，通常它为一个边界扫描链 JTAG，可使 CPU 进入调试模式，从而方便地进行断点设置、单步调试。
- M：Multiplier，快速乘法器，内置了硬件乘法器。包括 32 位 * 32 位 = 64 位和 32 位 * 32 位 + 32 位 = 64 位两种。V4 以上版本。
- I：Embedded ICE Logic，嵌入式跟踪宏单元，用于实现断点观测及变量观测的逻辑电路部分，其中的 TAP 控制器可接入到边界扫描链。

ARM 发展历程 1

- 1978 年 12 月 5 日，物理学家赫尔曼·豪泽（Hermann Hauser）和工程师 Chris Curry，在英国剑桥创办了 CPU 公司（Cambridge Processing Unit），主要业务是为当地市场供应电子设备。
- 1979 年，CPU 公司改名为 Acorn 计算机公司（艾康）。起初，Acorn 公司打算使用摩托罗拉公司的 16 位芯片，但是发现这种芯片太慢也太贵。他们转而向 Intel 公司索要 80286 芯片的设计资料，但是遭到拒绝，于是被迫自行研发。（目标：低价格、高性能、低功耗）
- 这个团队由 Roger Wilson 和 Steve Furber 带领，着手开发一种类似高级 6502 架构的处理器。1985 年，他们设计了他们自己的第一代 32 位、6M Hz 的处理器，用它做出了一台 RISC 指令集的计算机，简称 ARM（Acorn RISC Machine）。这就是 ARM 这个名字的由来。



Roger Wilson Steve Furber

“ARM 处理器系列”命名规则 1

- 命名格式：ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {-S} 12 个字段
- x：处理器系列，是共享相同硬件特性的一组处理器的具体实现，例如 ARM7TDMI、ARM740T 和 ARM720T 都属于 ARM7 系列。
- y：存储管理 / 保护单元。
 - 2：Cache + MMU
 - 3：改良型 MMU
 - 4：Cache + MPU
 - 6：无 Cache、MMU/MPU
- MMU：Memory Management Unit，内存管理单元，用来管理虚拟存储器、物理存储器的控制线路，同时也负责虚拟地址映射为物理地址。ARM 的 MMU 作为一个协处理器存在，编号为 15。
- MPU：Memory Protection Unit，内存保护单元，MPU 中一个域就是一些属性值及其对应的一片内存。这些属性包括：起始地址、长度、读写权限以及缓存等。

“ARM 处理器系列”命名规则 4

- 命名格式：ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {-S}
- E：DSP 增强指令（基于 TDMI）。
- J：Jazelle，Java 加速，这项技术允许直接执行 Java 字节代码，获得比基于软件的 Java 虚拟机 (JVM) 高得多的性能，比同等非 Java 加速核功耗降低 80%。
- F：向量浮点单元 VFP。
- S：可综合版本。可综合的，意味着处理器内核是以源代码形式提供的。这种源代码形式又可以被编译成一种易于 EDA 工具使用的形式。
- 说明：ARM7TDMI 之后的所有 ARM 内核，即使“ARM”标志后没有包含“TDMI”字符，也都默认包含了 TDMI 的功能特性；

ARM 发展历程 2

- 在 1980 年代晚期，苹果计算机开始与艾康计算机合作开发新版的 ARM 核心。由于该项目非常重要，艾康计算机于 1990 年将设计团队另组成一个名为 Advanced RISC Machines Ltd. 的新公司。基于这个原因，使得 ARM 更多时候反而称作 Advanced RISC Machine 而不是 Acorn RISC Machine。
- 1990 年 11 月 27 日，Acorn 公司正式改组为 ARM 计算机公司。苹果公司出资 150 万英镑，芯片厂商 VLSI 出资 25 万英镑，Acorn 本身则以 150 万英镑的知识产权和 12 名工程师入股。公司的办公地点非常简陋，就是一个谷仓。（ARM 公司是苹果、Acorn、VLSI、Technology 等公司的合资企业）



“ARM 处理器系列”命名规则 2

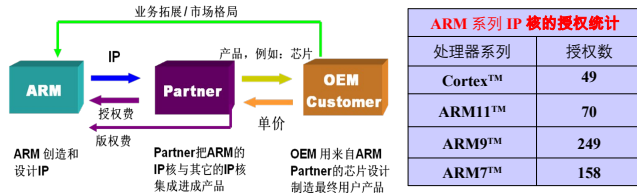
- 命名格式：ARM {x} {y} {z} {T} {D} {M} {I} {E} {J} {F} {-S}
- z：Cache。
 - 0：Cache Size variation 1（高速缓存大小变化 1）
 - 2：Cache Size variation 2（高速缓存大小变化 2）
 - 6：TCM（紧耦合内存）
- TCM：Tightly Coupled Memory，紧耦合内存，在 SOC 上用专用 BUS 与 CPU 连接的 Memory。由于是专用 BUS，可以高速访问。
- TCM 和 Cache 的区别：
 - (1) TCM 特定的高速空间，可以高速访问。
 - (2) TCM 具有物理地址。
 - (3) TCM 占用硬件资源比较少。
 - (4) 用途不同：TCM 保存常用到的数据，如中断向量表和实时数据，而 cache 只是为了加快处理器和外设之间的数据处理。

“ARM 体系结构”命名规则

- 命名格式：ARMv n Variants x(Variants)
- ARMv：固定字符，即 ARM Version。
- n：指令集版本号。比如，ARM 架构版本发布了 8 个系列，所以 n=[1:8]。其中最新的版本是第 8 版。
- Variants：变体。
- x(Variants)：排除 x 后面指定的变种。x 是固定字符，表示不支持括号内的变体。例如，ARMv5TxE 表示 ARM 指令集版本为 5，支持 T 变种，不支持 M 变种。
- 说明：ARM 处理器有时候还要区分架构版本、处理器核、处理器系列。例如，S3C2410：
 - 采用 ARMv4T 架构版本
 - 属于 ARM920T 处理器系列
 - 其处理器核为 ARM9TDMI

ARM 商业模式

- 商业模式：ARM 公司既不生产集成电路，也不销售集成电路，而是一个 IP（Intellectual Property）核设计公司。ARM 技术是以 IP 核的形式呈现出来的，ARM 公司通过把 IP 核授权给全球半导体公司使用，使得 ARM 技术得以传播和应用。
- ARM 核在每一项技术上不一定是最好的，但是在整体性能上却是最优的。
- 任何厂商都可以购买授权，最终使得 ARM 芯片遍地开花。



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 46

ARM 处理器技术进展 2

- MMU 技术。MMU 存储管理单元，用于管理存储系统。在嵌入式系统中，存储系统差别很大，可包含多种类型的存储器件，如 FLASH，SRAM，SDRAM 等，这些不同类型的存储器件的速度和位宽等各不相同。
- MPU 技术。MPU 内存保护单元中一个域就是一些属性值及其对应的一片内存。这些属性包括：起始地址、长度、读写权限以及缓存等。
- Branch Prediction 技术。分支预测技术。
- CoreSight 技术。内核扫描技术，内建于 ARM ETM 实时跟踪模块中，为完整的片上系统（SoC）设计提供最全面的调试、跟踪技术方案，通过最小端口可获得全面的系统可见度。
- ETM 技术。嵌入式跟踪宏单元（ETM，Embedded Trace Macrocell），进行代码的实时跟踪。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 49

智能手机

- 智能手机（Smartphone）：像个人电脑一样，具有独立的操作系统，可以由用户自行安装软件、游戏等第三方服务商提供的程序，通过此类程序来不断对手机的功能进行扩展，并可以通过移动通讯网络来实现无线网络接入的这样一类手机的总称。
- 智能手机体验：自行安装软件、游戏、GPS、WIFI、增强手机功能。

	六大智能手机厂商品牌	
品牌	主要支持操作系统	国家/地区
诺基亚	Symbian S60	芬兰
黑莓	BlackBerry OS	加拿大
摩托罗拉	Android, Linux+java	美国
苹果公司	iOS	美国
HTC	Windows Mobile, Android	中国台湾
Samsung	Bada, Android	南韩

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 52

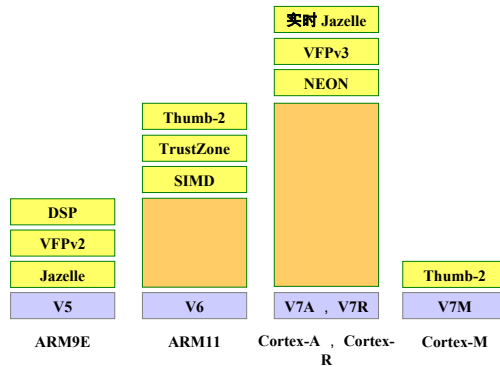
ARM 微处理器系列

- ARM 处理器当前有 6 个产品系列：ARM7、ARM9、ARM10、ARM11、SecurCore 和 Cortex。
- ARM7、ARM9、ARM10 和 ARM11 是 4 个通用处理器系列，每个系列提供一套特定的性能来满足设计者对功耗、性能和体积的需求。
- SecurCore 是第 5 个产品系列，是专门为安全设备而设计的。
- 来自于合作伙伴的产品（全球有 103 家大型 IT 公司采用 ARM 技术，20 家最大的半导体厂商中有 19 家是 ARM 的用户，包括德州仪器、意法半导体、Philips、Intel 等。）
- 例如：Intel XScale 微体系结构和 StrongARM 产品。



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 47

ARM 处理器技术进展 3



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 50

手机硬件的构成

- CPU——手机的大脑，中央处理器
- GPU——图形处理器，游戏必备
- RAM——运行内存，非常重要
- ROM——手机的闪存，放照片，视频，音乐，游戏
- 屏幕——视网膜屏幕（也称 Retina 显示屏）PPI 大于 300ppi
- 电池、相机、WIFI，蓝牙，陀螺仪，距离感应器。。。。。



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 53

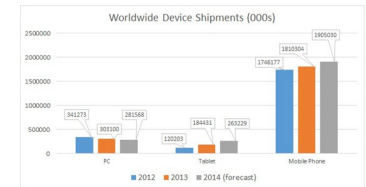
ARM 处理器技术进展 1

- ARM 指令集。ARM 指令体系具有 ARM、Thumb、Thumb-2 和 Thumb-2EE 四种指令集。
- 增强型 DSP 指令。
- Jazelle 技术。Java 加速器。
- SIMD 技术。单指令流多数据流，可同时进行 2 个 16 位操作数或 4 个 8 位操作数的运算。
- NEON 技术。NEON 用于流媒体处理的 SIMD 扩展，NEON 技术可加速多媒体和信号处理算法（如视频编码/解码、2D/3D 图形、游戏、音频和语音处理、图像处理技术、电话和声音合成），其性能至少为 ARMv5 性能的 3 倍，为 ARMv6 SIMD 性能的 2 倍。
- VFP 技术。VFP（Vector Floating-point Coprocessors）向量浮点运算是协处理器针对 ARM 架构的衍生技术。
- TrustZone 技术。可信区技术，是系统范围的安全方法，可以保护安全内存、加密块、键盘和屏幕等外设，从而可确保它们免遭软件攻击。其应用可拓展到电话银行、多媒体娱乐等广阔领域。TrustZone 技术是由硬件建构的访问控制方式，支持两颗虚拟的处理器。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 48

世界范围设备出货量

- 根据 Gartner（高德纳咨询公司，美国，全球最具权威的 IT 研究与顾问咨询公司）在 2013 年年底公布的报告，平板电脑在 2014 年将获得 43% 的增长，而手机也会增长 5%，而 PC 和笔记本的销量预计会再下滑 7%。
- Gartner 发布 2013 年对众多公司和组织机构具有战略意义的十大技术与趋势：移动设备、移动应用和 HTML5、个人云、企业应用商店、物联网、混合 IT 和云计算、战略大数据、可行性分析、内存计算、整合的生态系统。



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 51

ARM 支持的主流 OS 类型

- ARM 支持 OS 类型：Linux、Symbian、WP（微软公司，全球占有率 7%）、iOS（苹果公司，全球占有率 18.2%）、Android（谷歌公司，全球占有率 74.4%）。
- 2011 年 1 月 10 日微软公司宣布，下一版 Windows 将正式支持 ARM 处理器，即 Windows8 支持 ARM。
- 在移动设备市场，ARM 处理器的市场份额超过 90%，同时进军服务器市场，加上桌面电脑市场微软的支持，ARM 与 Intel 竞争还是合作，这是个问题。（ARM 的商业模式是开放的，任何厂商都可以购买授权，所以可能并不是 Intel vs. ARM，而是 Intel vs. 世界上所有其他半导体公司。）



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 54

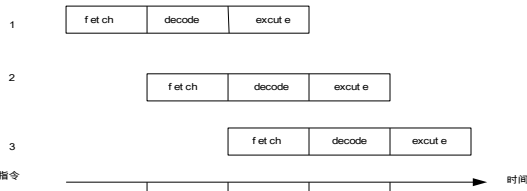
架构	厂商	CPU	主频	GPU	代表机型
Cortex-A8	高通	QSD8250	1GHz	adreno 200	HTC Desire
		MSM8255	1GHz	adreno 205	HTC Desire S
	德州仪器	OMAP3430	720MHz	PowerVR SGX530	MOTO 里程碑
		OMAP3630	720MHz (1GHz)	PowerVR SGX530	MOTO 里程碑 2、ME811
	三星	SSPC110	1GHz	PowerVR SGX540	三星 I9000
Cortex-A9	苹果	A4	1GHz	PowerVR SGX535	iPhone4、iPad
	nVIDIA	Tegra 2	双核 1GHz	Geforce ULV	ME860、XOOM
	高通	MSM8260	双核 1GHz 以上	adreno 220	HTC Sensation
	德州仪器	OMAP4430	双核 1GHz 以上	PowerVR SGX540	黑莓 Playbook
	三星	Exynos	双核 1GHz	ARM Mali 400MP	GALAXY S II
	苹果	A5	双核 1GHz	PowerVR SGX543MP	iPad2

RISC 与 CISC 技术比较

指 标	RISC	CISC
指令集	一个周期执行一条指令，通过简单指令的组合实现复杂操作指令长度固定。	指令长度不固定，执行需要多个周期。
流水线	流水线每周期前进一步。	指令的执行需要调用微代码的一个微程序。
寄存器	更多通用寄存器。	用于特定目的的专用寄存器。
Load/Store 结构	独立的 Load 和 Store 指令完成数据在寄存器和外部存储器之间的传输。	处理器能够直接处理存储器中的数据。

ARM 的 3 级流水线

- ARM7 架构采用了一个 3 段的流水线：
 - (1) 取指：将指令从内存中取出来。
 - (2) 译码：操作码和操作数被译码以决定执行什么功能。
 - (3) 执行：执行已译码的指令。



- 2013 年上半年，国内市场手机出货量达 2.91 亿部，同比增长 44.7%，持续保持快速增长势头。其中，智能手机出货量达到 2.14 亿部，同比增长 120.5%，市场份额达到 73.7%。

2013 年第二季度中国智能手机出货量

手机品牌	出货量（万台）	市场占比
三星	1550	17.6%
联想	1080	12.3%
宇龙酷派	1070	12.2%
中兴 ZTE	770	8.7%
华为	750	8.6%
小米	440	5%
苹果	430	4.8%
其它	2710	30.8

ARM 体系结构采用的特别技术

- ARM 体系结构还采用了一些特别的技术：
 - 所有的指令都可根据前面的执行结果决定是否被执行，提高了指令的执行效率。
 - 可用 Load/Store 指令批量传输数据，以提高数据的传输效率。
 - 可在一条数据处理指令中同时完成逻辑处理和移位处理。
- RISC 和 CISC 各有优势，界限并不那么明显。
- 现代的 CPU 往往采用 CISC 的外围，内部加入了 RISC 的特性，如超常指令集 CPU 就是融合了 RISC 和 CISC 的优势，成为未来的 CPU 发展方向之一。

ARM 的 5 级流水线

- ARM9 和 StrongARM 架构都采用了 5 级流水线。
- 增加了 I-Cache 和 D-Cache，把存储器的取指与数据存取分开。
- 增加了数据写回的专门通路和寄存器。
- 5 级流水线把指令的执行过程分割为 5 部分而不是 3 部分，这就减少了在每个时钟周期内必须完成的最大工作量，进而允许使用较高的时钟频率。
- 5 级流水线可分为：
 - (1) 取指：将指令从指令存储器中取出，放入指令流水线中。
 - (2) 指令译码：对指令进行译码，从寄存器堆中读取寄存器操作数。
 - (3) 执行：将一个操作数移位，产生 ALU 结果。如果指令是 load 或 store，在 ALU 中计算存储器的地址。
 - (4) 数据缓存：如果需要，访问数据存储器；否则，ALU 的结果只是简单地缓冲一个时钟周期，以便使所有指令具有同样的流水线流程。
 - (5) 写回：将指令产生的结果回写到寄存器堆，包括任何从存储器读取的数据。

- 2.2.1 RISC 体系结构
- CPU 从指令集的特点上可以分为两类：CISC 和 RISC。
- CISC：复杂指令集计算机。随着计算机技术的发展而不断引入新的复杂的指令集，计算机的体系结构会越来越复杂。
- 据统计，CISC 指令中，大约有 20% 指令会被反复使用，占整个程序代码的 80%，而余下的 80% 指令却不经常使用，在程序设计中只占 20%。
- RISC：精简指令集计算机。采用固定长度指令格式。基本特点如下：
 - (1) 大多数指令只需要执行简单和基本的功能，其执行过程在一个机器周期内完成。
 - (2) 只保留加载/存储指令。操作数由加载/存储指令从存储器取出放寄存器内操作。
 - (3) 芯片逻辑不采用或少采用微码技术，而采用硬布线逻辑。
 - (4) 减少指令数和寻址方式。
 - (5) 指令格式固定，指令译码简化。
 - (6) 优化编译。

流水线技术

- 流水线技术：是一种将每条指令分解为多步，并让各步操作重叠，从而实现几条指令并行处理的技术。
- 流水线技术是在本质上影响程序执行速度的因素。
- 由于计算机中一条指令的各个执行阶段相对独立，因此，现代 CPU 大多设计成流水线型的机器，在这种类型机器中几个指令可以并行执行。采用流水线的重叠技术大大提高了 CPU 的运行效率。
- 当流水线内部的信息通畅流动时，CPU 流水线能够工作得最好。
- 但实际应用中，指令各执行阶段的操作时间长短不同，有一些指令序列可能会打断流水线内的信息流，所以有时流水线操作不十分通畅，会暂时降低 CPU 的执行速度。



ARM7 与 ARM9 流水线比较

- 虽然 ARM7 与 ARM9 的流水线条数不同，但使用了相同的流水线执行机制，提高了兼容性。由图可见，两者的指令执行阶段都处于第三级，所以都是：PC= 当前执行指令地址 +8。
- 假设当前正在执行 0 地址处的指令。

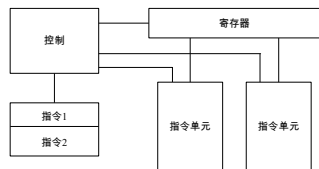


- 在 ARM7 的 3 级流水线结构中，流水线易出现阻塞或间断，这必然会降低流水线的效率，因此为了提高微处理器的性能，必须考虑如何优化微处理器的组织结构。

- 解决方法：**缩短程序执行时间、解决流水线相关问题。

超标量执行

- 超标量技术：**通过重复设置多套指令执行部件，同时处理并完成多条指令，实现并行操作，来达到提高处理速度的目的。
- 所有 ARM 内核，如 ARM7、ARM9 和 ARM11 等，都是单周期指令机。
- ARM 公司下一代处理器将是每周能处理多重指令的超标量机。
- 超标量处理机：**一个时钟周期内同时执行多条指令的处理机。
- 超标量与流水线技术是兼容的，为了能够在一个时钟周期内同时发射多条指令，**超标量处理机必须有两条或两条以上能够同时工作的指令流水线。**但与此同时，也带来了多流水线的调度问题和操作部件的资源冲突问题。
- 超标量处理器在执行的过程中必须**动态地检查指令相关性**。
- 如果代码中有分支指令，必须将分支被执行和分支不被执行这两种情况分开考虑。
- 计算执行时间几乎是不可能的。



2.2.4 ARM 微处理器的应用选型

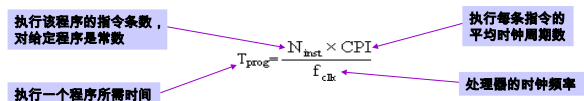
- 从应用的角度出发，选择 ARM 微处理器时应考虑一些问题。
- 1. ARM 微处理器内核的选择。**
 - 运行 Windows CE、Linux 等：ARM720T 以上带有 MMU 的处理器，ARM720T、ARM920T、ARM922T、ARM946T、Xscale
 - 运行 uClinux、RTLinux、uC/OS - II 等：ARM7TDMI
- 2. 系统的工作频率。**

系统的工作频率在很大程度上决定了 ARM 微处理器的处理能力。

 - ARM7 系列：典型处理速度 0.9MIPS/MHz，常见系统主时钟为 20MHz-133MHz。
 - ARM9 系列：典型处理速度 1.1MIPS/MHz，常见系统主时钟频率为 100MHz-233MHz。
 - ARM10 最高可以达到 700MHz。

不同芯片对时钟的处理不同，有的芯片只需要一个主时钟频率，有的芯片内部时钟控制器可以分别为 ARM 核和 USB、UART、DSP、音频等功能部件提供不同频率的时钟。

- 计算微处理器运行一个给定程序所需时间的公式如下：



- 缩短程序执行时间，提高处理器执行的性能，有两种方法：

- 提高时钟频率 fclk。**这意味着处理器时钟周期缩短，能完成的工作量要比原来少（每个步骤都需要一定的时间，时间少了，步骤就只能减少）。这就要求简化流水线每一级的逻辑，因而流水线的级数就要增加。
- 减少每条指令的平均时钟周期数 CPI。**要减少 CPI，即提高每个时钟周期流水线中指令的吞吐量（很多时候不能保证一个时钟周期完成一条指令，因为存在多周期指令），就必须改善 3 级流水线中的阻塞状况。这就需要**解决流水线的相关问题**。

2.2.2 ARM 微处理器的寄存器结构

- ARM 处理器共有 **37 个**寄存器，被分为若干个组（BANK），这些寄存器包括：
 - 31 个通用寄存器，包括程序计数器（PC 指针），均为 32 位的寄存器。**
 - 6 个状态寄存器，用以标识 CPU 的工作状态及程序的运行状态，均为 32 位，目前只使用了其中的一部分。**
- ARM 处理器又有 **7 种**不同的处理器模式，在每一种处理器模式下均有一组相应的寄存器与之对应。
- 在任意一种处理器模式下，可访问的寄存器包括 **15 个**通用寄存器（R0 ~ R14）、**1~2 个**状态寄存器和程序计数器。
- 有些寄存器是 7 种处理器模式共用的，还有些寄存器则是针对不同的处理器模式的。
- 后续详解。

ARM 微处理器的应用选型

- 3. 芯片内存存储器的容量。**

多数 ARM 微处理器片内存存储器的容量都不大，需要用户在设计系统时外扩存储器。也有部分芯片具有相对较大的片内存存储空间，如 **ATMEL 的 AT91F40162** 就具有高达 2MB 的片内存存储空间，用户在设计时可考虑选用这种类型，以简化系统的设计。
- 4. 片内外围电路的选择。**

几乎所有的 ARM 芯片均根据各自不同的应用领域，扩展了相关功能模块，并集成在芯片之中，称之为**片内外围电路**。如 USART（UART/SCI）、IIC、SPI、USB（HOST/DEVICE）、IrDA、Ethernet（MAC）、IIS/AC97、LCD、Keypad、RTC、ADC、DAC、DSP 协处理器等。
- 5. 生产商的选择。**如高通、三星电子、英伟达、德州仪器、VLSI、Intel、IBM、飞思卡尔、atmel、LG、NEC、SONY、富士通、英飞凌、任天堂、恩智浦半导体、OKI 电气工业、Sharp 等公司。

- 流水线中的相关问题：**结构相关、数据相关、控制相关。
- 结构相关：**某些指令在流水线中重叠执行时，产生资源冲突。
措施：1）采用分离式指令 Cache 和数据 Cache。
2）ALU 中采用单独加法器来完成地址计算。
- 数据相关：**当一条指令需要前面指令的执行结果，而这些指令均在流水线中重叠执行时，就可能引起流水线的**数据相关**。
- 数据相关有“写后读”、“写后写”和“读后写”等。**
措施：1）旁路技术。
2）流水线互锁技术。发生互锁时，流水线停止这个指令的执行。
- 控制相关：**当流水线遇到分支指令和其他会改变 PC 值的指令时，就会发生控制相关。
措施：1）引入延时分支。
2）尽早计算出分支转移成功时的 PC 值（即分支的目标地址）。

2.2.3 ARM 微处理器的指令结构

- ARM 微处理器在较新的体系结构中支持两种指令集：**ARM 指令集**和**Thumb 指令集**。
 - ARM 指令为 32 位的长度**
 - Thumb 指令为 16 位长度**
- Thumb 指令集为 ARM 指令集的功能子集，但与等价的 ARM 代码相比较，可节省 30% ~ 40% 以上的存储空间，同时具备 32 位代码的所有优点。**
- 后续详解。

2.3 ARM 微处理器的工作状态

- V4 版以后有：**
 - 32 位 ARM 指令集。**
 - 16 位 Thumb 指令集，功能是 ARM 指令集的功能子集。**
- ARM7TDMI 核以后，T 变种的 ARM 微处理器有**两种工作状态**：
 - ARM 状态。**
 - Thumb 状态。**
- 当 ARM 微处理器执行 32 位的 ARM 指令集时，工作在 ARM 状态。
- 当 ARM 微处理器执行 16 位的 Thumb 指令集时，工作在 Thumb 状态。



ARM 与 Thumb 状态转换

- ARM 指令集和 Thumb 指令集均有切换处理器状态的指令，在程序的执行过程中，微处理器可以随时在两种工作状态之间切换，这种切换不影响处理器的工作模式和相应寄存器中的内容。
- ARM 微处理器总是在 ARM 状态下开始执行代码。由一种工作状态切换到另一种工作状态的方法如下。
- 进入 Thumb 状态：**
 - (1) 当操作数寄存器的位 [0]=1 时，执行 BX 指令，使微处理器从 ARM 状态切换到 Thumb 状态。
 - (2) 在处理器处于 Thumb 状态时，如果发生异常（如 IRQ、FIQ、Undef、Abort、SWI 等），则异常处理返回时自动切换到 Thumb 状态。
- 进入 ARM 状态：**
 - (1) 当操作数寄存器的位 [0]=0 时，执行 BX 指令，可以使微处理器从 Thumb 状态切换到 ARM 状态。
 - (2) 在进行异常处理时，把 PC 指针放入异常模式链接寄存器中，并从异常向量地址开始执行程序，也可以使处理器切换到 ARM 状态。

ARM 数据存储格式 - 小端模式

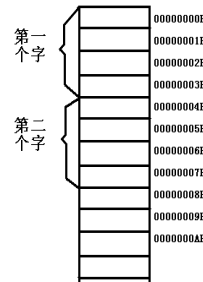
- 小端模式：**高字节存放在高存储器地址，低字节存放在低存储器地址。
- 存储器连续的 4 个字节空间 (A, A+1, A+2, A+3) 为一个字空间（起始地址为 4 的倍数）
- 字在存储器中的保存规则：
 - B0 存入 A 地址（最低字节→最低地址）
 - B1 存入 A+1 地址
 - B2 存入 A+2 地址
 - B3 存入 A+3 地址（最高字节→最高地址）

MMU 的页式存储管理

- ARM920T 的 MMU 采用分页虚拟存储管理方式。它把虚拟存储空间分成固定大小的页，把物理存储空间也分成同样大小的页。
- 通过查询页表，实现虚地址到物理地址的转换。
- 由于页表存放在主存中，查询页表花费的代价很大，因此，通常采用快表技术（TLB）来提高地址变换效率。
- 页表：**位于内存中，每个页表项对应于虚拟存储空间的一页，包含该虚拟页对应的物理页的地址（也就是虚页到实页）、访问权限、缓冲特性等。
- 在 ARM 系统中，使用协处理器 CP15 的寄存器 C2 来保存一级页表的基地址。
- 快表 TLB：** Translation Lookaside Buffer，小容量（通常为 8 ~ 16 字）、高速度（访问速度和 CPU 中通用寄存器相当）的存储部件，存放当前访问需要的地址变换目标。

2.4 ARM 体系结构的存储器格式

- 存储空间：**地址线 32 条，4GB (2^{32} 存储字节)。
- 存储基本单位：**字节。
- 存储组织：**字。
- 数据存储格式有两种：**大端模式、小端模式。



2.4.1 指令长度及数据类型

- ARM 处理器指令长度是 32 位（ARM 状态）和 16 位（Thumb 状态）。
- ARM 存储数据类型：六种数据类型：**
 - (1) 8 位有符号和无符号字节。
 - (2) 16 位有符号和无符号半字，以 2 字节的边界对齐。
 - (3) 32 位有符号和无符号字，以 4 字节的边界对齐。
- 在内部，所有 ARM 操作都是面向 32 位的操作数，只有数据传送指令支持较短的字节和半字的数据类型。
- 对齐要求：**
 - (1) ARM 指令都是 32 位的字，必须以字（4 字节）为单位边界对齐。
 - (2) Thumb 指令是 16 位半字，必须以 2 字节为单位边界对齐。
 - (3) 当从存储器调入一个字节和半字时，根据指令对数据的操作类型，将其无符号 0 或有符号“符号位”扩展为 32 位，进而作为 32 位数据在内部进行处理。

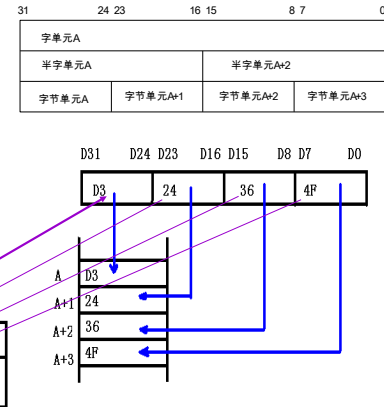
协处理器 CP15

- 协处理器 CP15 的主要管理和配置功能：
 - (1) 页表
 - (2) 快表 TLB
 - (3) 域、访问权限
 - (4) cache 和写缓冲器
 - (5) 快速上下文切换

CP15 中与 MMU 相关的寄存器	作用
寄存器 C1 中某些位	使能 / 禁止 MMU、cache、写缓冲器
寄存器 C2	保存一级页表的基地址
寄存器 C3	设置域（domain）的访问权限
寄存器 C4	保留
寄存器 C5	主存访问失效状态指示
寄存器 C6	主存访问失效时的地址
寄存器 C8	控制与清除 TLB 内容
寄存器 C10	控制与锁定 TLB 内容相关的操作

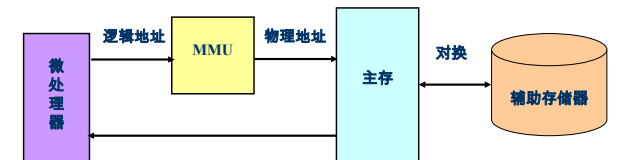
ARM 数据存储格式 - 大端模式

- 大端模式：**高字节存放在低存储器地址，低字节存放在高存储器地址。
- 存储器连续的 4 个字节空间 (A, A+1, A+2, A+3) 为一个字空间（起始地址为 4 的倍数）
- 字在存储器中的保存规则：
 - B3 存入 A 地址（最高字节→最低地址）
 - B2 存入 A+1 地址
 - B1 存入 A+2 地址
 - B0 存入 A+3 地址（最低字节→最高地址）



2.4.2 存储管理单元 (MMU)

- 存储管理单元 (MMU)：**用于在 CPU 和物理内存之间进行地址转换。
- 内存映射：**将地址从逻辑空间映射到物理空间的过程。
- 在 ARM 系统中，存储管理单元 (MMU) 主要完成的工作：
 - (1) 将虚地址转换成物理地址。
 - (2) 控制内存的访问权限。
- MMU 关闭时，虚地址直接输出到物理地址总线。
- 虚拟地址存储系统示意图。

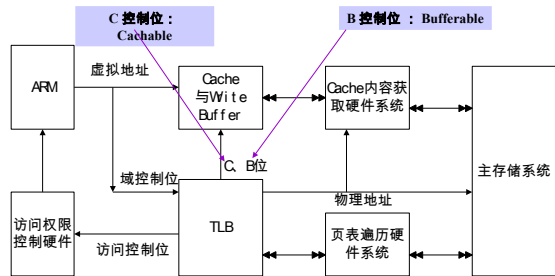


使能 MMU

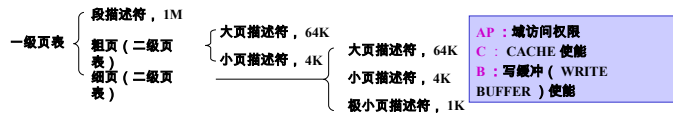
- 协处理器 CP15 的寄存器 C1 的位 [0] 用于设置禁止 / 使能 MMU。
 - C1 位 [0]=0 时，禁止 MMU；C1[0]=1 时，使能 MMU。
- 下面指令实现使能 MMU：
 - MRC P15, 0, R0, C1, 0, 0 ; C1 的内容赋给 R0
 - ORR R0, #0x1
 - MCR P15, 0, R0, C1, 0, 0 ; R0 的内容赋给 C1
- 使能 MMU 时，其控制存储访问的过程是：
 - (1) 首先在 TLB 中查找虚拟地址。
 - (2) 如果该虚拟地址对应的地址变换目标不在 TLB 中，则到页表中查询对应的页表项，并把查询到的结果添加到 TLB 中。
 - (3) 如果 TLB 已满，还需根据一定的淘汰算法进行替换。得到地址变换目标后，进行一下步骤的操作。

禁止 MMU

- 禁止 MMU 时，存储器按如下处理：
 - (1) 根据具体情况，确定是否允许 cache 和写缓冲器。
 - (2) 没有存储器的访问权限，MMU 也不会产生异常信号。
 - (3) 虚地址就是物理地址，此时使用平板存储模式。



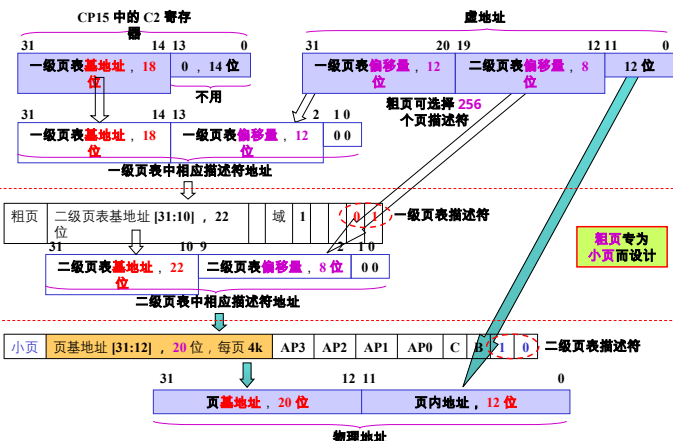
页表、描述符



一级页表：段、粗页、细页描述符															
类型	31	20	19	12	11	10	9	8	5	4	3	2	1	0	
无效															0
粗页															1
段															1
细页															1

二级页表（粗页）：大页、小页描述符															
类型	31	16	15	12	11	10	9	8	7	6	5	4	3	2	1
无效															0
大页															1
小页															1
极小															1

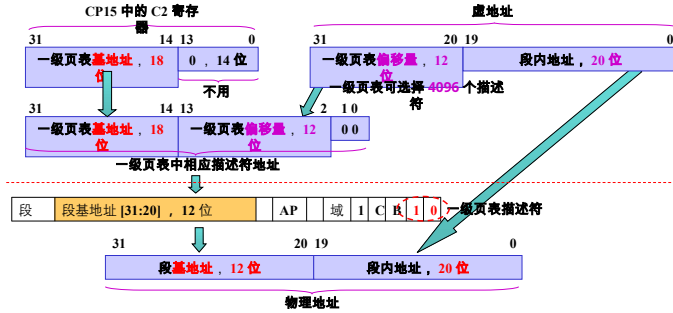
粗页二级页表地址变换 - 小页 4K



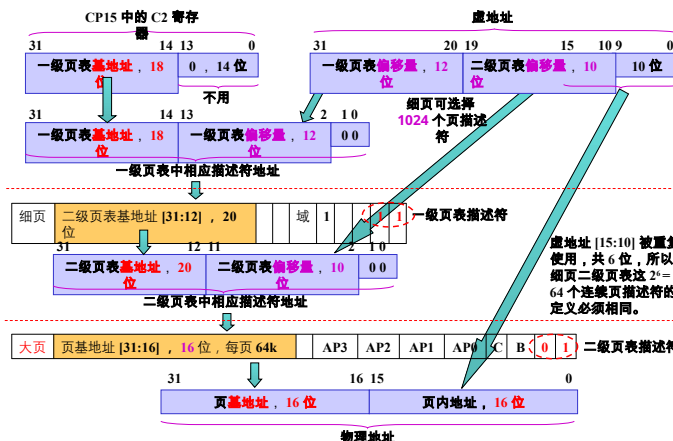
MMU 中的地址变换过程

- 虚地址到物理地址的映射以内存块为单位进行。
- MMU 的地址变换过程通过两级页表实现。
- 一级页表：包含段描述符、粗页描述符、细页描述符。二级页表有粗页、细页 2 种形式。以段为单位的地址变换只需要一级页表。
- 二级页表：包含大页、小页、极小页的描述符。以页为单位的地址变换过程需要二级页表。
- ARM 920T 支持的存储块大小有 4 种：
 - (1) 段 (Section)：以 1MB 为存储块单位。
 - (2) 大页 (Large Pages)：以 64KB 为存储块单位。
 - (3) 小页 (Small Pages)：以 4KB 为存储块单位。
 - (4) 极小页 (Tiny Pages)：以 1KB 为存储块单位。

段地址变换

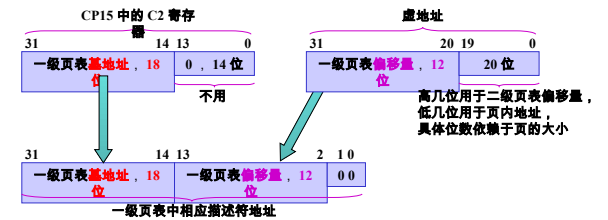


细页二级页表地址变换 - 大页 64K

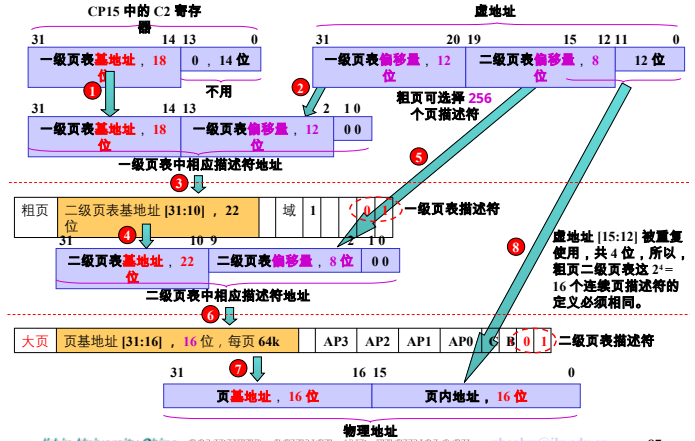


一级页表地址变换

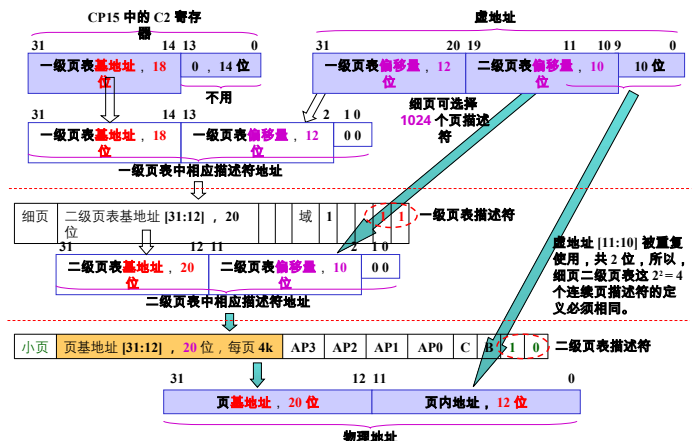
- CP15 的寄存器 C2：存放一级页表的基地址，32 位，高 18 位有效。
- 一级页表地址形成：
$$\text{一级页表基地址 (18 位)} + \text{一级页表偏移量 (12 位)} + 00 \text{ (字对齐)}$$
$$= \text{CP15-C2 高 18 位} + \text{虚地址高 12 位} + 00$$
- 一级页表中有 $2^{12} = 4096 = 4K$ 个描述符。



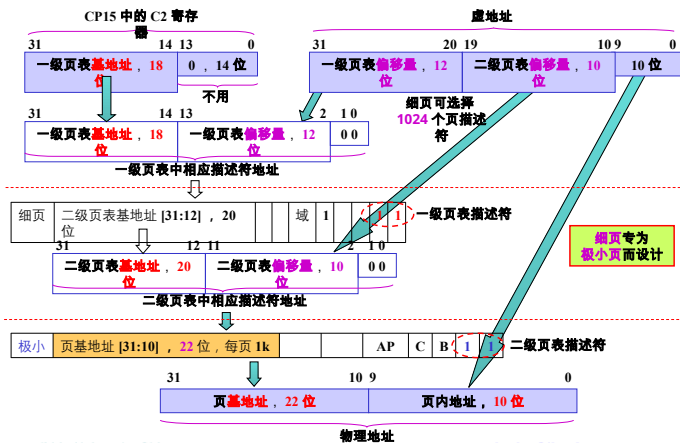
粗页二级页表地址变换 - 大页 64K



细页二级页表地址变换 - 小页 4K



页二级页表地址变换 - 极小页 1K



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 91

MMU 中的存储访问权限控制

- CP15 寄存器 C1 中：
 - S：修改 MMU SYSTEM
 - R：ROM 保护状态
 - AP：Access permissions，访问权限。
- S、R、AP 联合作用控制存储访问权限。

AP	S	R	特权级时访问权限	用户级时访问权限
00	0	0	没有访问权限	没有访问权限
00	1	0	只读	没有访问权限
00	0	1	只读	只读
00	1	1	不可预知	不可预知
01	X	X	读 / 写	没有访问权限
10	X	X	读 / 写	只读
11	X	X	读 / 写	读 / 写

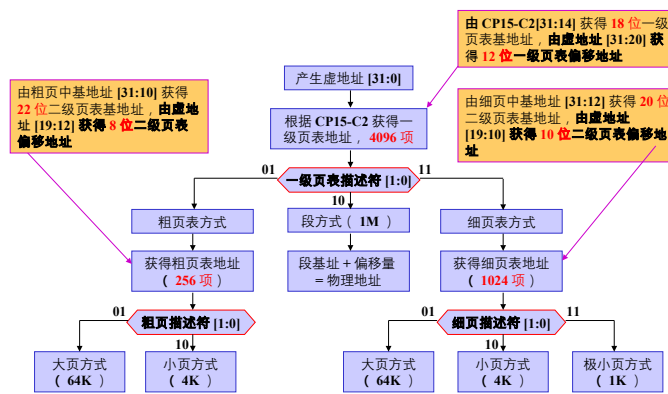
Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 94

存储访问失效 1

- 在 ARM 920T 中，MMU 的存储访问失效有 4 种类型：地址对齐失效、地址变换失效、域控制失效、访问权限控制失效。
- 当发生存储访问失效时，存储系统可以中止 3 种存储访问：cache 内容预取、非缓冲的存储器访问操作、页表访问。
- 检测存储访问失效的 2 种机制：
 - (1) MMU 失效机制：当 MMU 检测到存储访问失效时，它可以向微处理器报告该情况，并将存储访问失效的相关信息保存到寄存器 C5 和 C6 中。
 - (2) 外部存储访问中止机制：存储系统向微处理器报告存储访问失效。
- 上述两种情况通称为存储访问中止 (abort)。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 97

MMU 地址变换流程



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 92

MMU 中的域

- MMU 中的域指的是是一些段、大页或者小页的集合。
- ARM 920T 支持最多 16 个域，每个域的访问控制特性都是由 CP15 中的寄存器 C3 中的两位来控制的。
- CP15 中寄存器 C3：每两位控制一个域的访问控制特性。

CP15-C3	域 15	域 14	域 13	域 12	域 11	...	...	域 4	域 3	域 2	域 1	域 0
---------	------	------	------	------	------	-----	-----	-----	-----	-----	-----	-----

控制位编码	访问类型	含 义
00	没有访问权限	这时访问该域将产生访问失效
01	客户类型	根据页表中地址变换条目中的访问权限控制位决定时是否允许特定的存储访问
10	保留	使用该值会产生不可预知的结果
11	管理者权限	不考虑页表中地址变换条目中的访问权限控制位，这种情况下不会产生访问失效

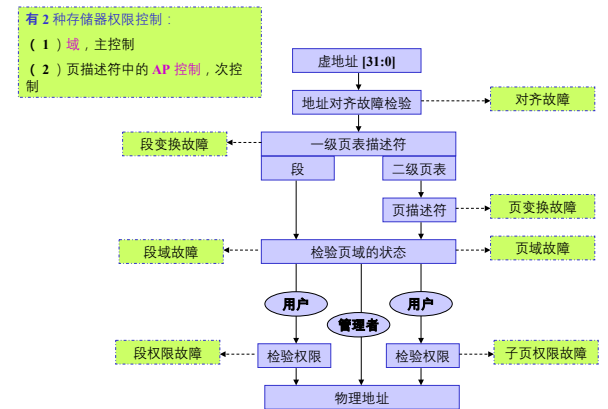
Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 95

存储访问失效 2

- 如果存储访问发生在数据访问周期，微处理器将产生数据访问中止异常。
- 如果存储访问发生在指令预取周期，当该指令执行时，微处理器将产生指令预取异常。
- MMU 中与存储访问失效相关的寄存器有两个：C5 和 C6。
- C5 为失效状态寄存器，C6 为失效地址寄存器。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 98

存储权限



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 93

快表 TLB 操作

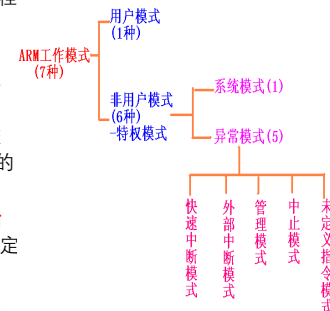
- 从虚拟地址到物理地址的变换过程其实就是查询页表的过程，由于页表存放在主存储器中，这个查询代价很大。而程序在执行时其过程具有局部性，对页表中各存储单元的访问并不是随机的，在一段时间内，只局限在少数几个单元中。
- 因此，采用 TLB 技术可以提高存储系统的整体性能。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 96

2.5 处理器模式

- ARM 微处理器支持 7 种工作模式：
 - 处理器的各种工作模式由当前程序状态寄存器 CPSR 的低 5 位 M[4:0] 决定。
 - 不同的工作模式间可以相互切换，有两种方式：

- (1) 发生异常时，处理器自动改变 CPSR 中 M[4:0] 的值，进入相应的工作模式；
- (2) 处理器处于特权模式时，用指令向 CPSR 的 M[4:0] 字段写入特定的值，进入相应的工作模式。
- 用户模式时，不能改变工作模式，除非发生异常。



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 99

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 100

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 100

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 100

- Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 100

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 100

		圖 1 學子更次山科上午練習
--	--	----------------

		圖 1 學子更次山科上午練習
--	--	----------------

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 101



		各模式下实际访问的寄存器
--	--	--------------

寄存器类型	寄存器在汇编中的名称	寄存器 1	寄存器 2	寄存器 3	寄存器 4	寄存器 5	寄存器 6	寄存器 7
通用寄存器	RAX, RBX, RCX, RDX, RDI, RSI, RBP, RSP, R8, R9, R10, R11, R12, R13, R14, R15							
段寄存器	CS, DS, SS, ES, FS, GS							
控制寄存器	CR0, CR2, CR3, CR4, CR8							
调试寄存器	DR0, DR1, DR2, DR3, DR6, DR7							
浮点寄存器	XMM0, XMM1, XMM2, XMM3, XMM4, XMM5, XMM6, XMM7, XMM8, XMM9, XMM10, XMM11, XMM12, XMM13, XMM14, XMM15							
MMX寄存器	K0, K1, K2, K3, K4, K5, K6, K7							
AVX寄存器	ZMM0, ZMM1, ZMM2, ZMM3, ZMM4, ZMM5, ZMM6, ZMM7, ZMM8, ZMM9, ZMM10, ZMM11, ZMM12, ZMM13, ZMM14, ZMM15							

各機子平均回分回社率表		

寄存器	寄存器在汇编	各模式下实际访问的寄存器					
		ARMv7-M	ARMv7-M	ARMv7-M	ARMv7-M	ARMv7-M	ARMv7-M

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 102

- Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 102

各模式下实际计价的要素		
模式一	模式二	模式三

寄存器	寄存器在汇编	各模式下实际访问的寄存器					

各機手工作區及包井圖		

寄存器	寄存器在汇编	各模式下实际访问的寄存器				其中 R0 ~ R7 为 未
-----	--------	--------------	--	--	--	-----------------------

R8 ~ R14 通用寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)				R0			
	R1(a2)				R1			
	R2(a3)				R2			
	R3(a4)				R3			
	R4(v1)				R4			
	R5(v2)				R5			
	R6(v3)				R6			
	R7(v4)							
	R8(v5)				R8			R8_fiq
	R9(SB,v6)				R9			R9_fiq
	R10(SL,v7)				R10			R10_fiq
	R11(FP,v8)				R11			R11_fiq
	R12(IP)				R12			R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)				R15			
状态寄存器	CPSR				CPSR			

寄存器 R8 ~ R14 为分组寄存器。它们所对应的物理寄存器取决于当前的处理器模式，几乎所有允许使用通用寄存器的指令都允许使用分组寄存器。

R8 ~ R12 的两个分组

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)				R0			
	R1(a2)				R1			
	R2(a3)				R2			
	R3(a4)				R3			
	R4(v1)				R4			
	R5(v2)				R5			
	R6(v3)				R6			
	R7(v4)							
	R8(v5)				R8			R8_fiq
	R9(SB,v6)				R9			R9_fiq
	R10(SL,v7)				R10			R10_fiq
	R11(FP,v8)				R11			R11_fiq
	R12(IP)				R12			R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)				R15			
状态寄存器	CPSR				CPSR			

寄存器 R8 ~ R12 有两个分组的物理寄存器。一个用于除 FIQ 模式之外的所有寄存器模式，另一个用于 FIQ 模式。这样在发生 FIQ 中断后，可以加速 FIQ 的处理速度。

R13、R14 的 6 个分组

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)				R0			
	R1(a2)				R1			
	R2(a3)				R2			
	R3(a4)				R3			
	R4(v1)				R4			
	R5(v2)				R5			
	R6(v3)				R6			
	R7(v4)							
	R8(v5)				R8			R8_fiq
	R9(SB,v6)				R9			R9_fiq
	R10(SL,v7)				R10			R10_fiq
	R11(FP,v8)				R11			R11_fiq
	R12(IP)							
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)				R15			
状态寄存器	CPSR				CPSR			

寄存器 R13、R14 分别有 6 个分组的物理寄存器。一个用于用户和系统模式，其余 5 个分别用于 5 种异常模式。

堆栈指针寄存器 R13 (SP)

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)				R0			
	R1(a2)				R1			
	R2(a3)				R2			
	R3(a4)				R3			
	R4(v1)				R4			
	R5(v2)							
	R6(v3)							
	R7(v4)							
	R8(v5)							R8_fiq
	R9(SB,v6)				R9			R9_fiq
	R10(SL,v7)				R10			R10_fiq
	R11(FP,v8)				R11			R11_fiq
	R12(IP)				R12			R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)				R15			
状态寄存器	CPSR				CPSR			

寄存器 R13 常作为堆栈指针 (SP)。在 ARM 指令集中，没有以特殊方式使用 R13 的指令或其它功能，只是习惯上都这样使用。但是在 Thumb 指令集中有必须使用 R13 的指令。

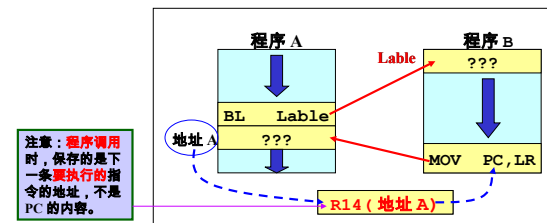
链接寄存器 R14 (LR)

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)				R0			
	R1(a2)				R1			
	R2(a3)				R2			
	R3(a4)				R3			
	R4(v1)				R4			
	R5(v2)				R5			
	R6(v3)				R6			
	R7(v4)							
	R8(v5)							
	R9(SB,v6)							
	R10(SL,v7)							
	R11(FP,v8)							
	R12(IP)							
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)				R15			
状态寄存器	CPSR				CPSR			

R14 为链接寄存器 (LR)，在结构上有两个特殊功能：
• 在每种模式下，模式自身的 R14 用于保存子程序返回地址；
• 当发生异常时，将异常模式对应的 R14 设置为异常返回地址 (有些异常有一个小的固定偏移量)。

R14 (LR) 的作用

- 作用：保存程序断点地址 (返回地址)，称作 LR，链接寄存器。
- 例如：
 - 程序 A 执行过程中调用程序 B；
 - 程序跳转至标号 Lable，执行程序 B。同时硬件将“BL Lable”指令的下一条指令所在地址存入 R14 (LR)；
 - 程序 B 执行最后，将 R14 寄存器的内容放入 PC，返回程序 A；



注意：程序调用时，保存的是下一条要执行的指令的地址，不是 PC 的内容。

R14 (LR) 的作用示例

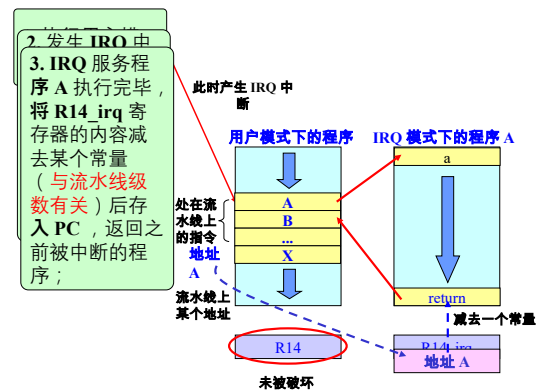
```

BL    SUB1                ; 子程序调用
.....
.....
SUB1
    STMFD SP!,{<regs>, LR} ; 将 R14 存入堆栈
    .....
    LDMFD SP!,{<regs>, PC} ; 完成子程序返回
    
```

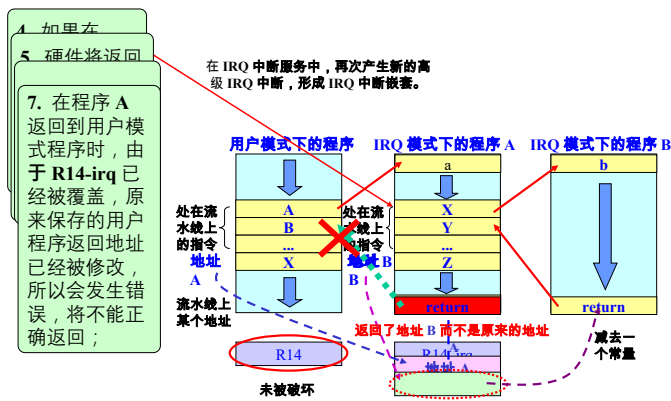
R14 寄存器与异常发生

- 异常发生时，程序要跳转至异常服务程序，对返回地址的处理与子程序调用类似，都是由硬件完成的。
- 区别：有些异常有一个小的偏移常量。
- R14 寄存器注意要点：
 - 当发生异常嵌套时，这些异常之间可能会发生冲突。
 - 例如：如果用户在用户模式下执行程序时发生了 IRQ 中断，用户模式寄存器不会被破坏。但是如果允许在 IRQ 模式下的中断处理程序重新使能 IRQ 中断，并且发生了嵌套的 IRQ 中断时，外部中断处理程序保存在 R14_irq 中的任何值都将被嵌套中断的返回地址所覆盖。

用户模式下产生 IRQ 中断



用户模式下产生 IRQ 中断嵌套



程序计数器 R15 (PC)

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器							
		用户	系统	管理	中止	未定义	中断	快中断	
通用寄存器和程序计数器	- 寄存器 R15 为 程序计数器 (PC)，它指向正在取指的地址。可以认为它是一个通用寄存器，但是对于它的使用有许多与指令相关的限制或特殊情况。 - ARM7 正常操作时，从 R15 读取的值是处理器正在取指的地址，即 $R15 (PC) = \text{当前正在执行指令的地址} + 8$ 注：1.8 是指 8 个字节，是两条 ARM 指令的长度。 2. ARM 是 3 级流水线：取指，译码，执行。 - 由于 ARM7 指令总是以字为单位，所以 R15 寄存器的最低两位总是为 0。 - R15 值的改变将引起程序执行顺序的改变。 - 向 R15 内写入一个值，程序将跳转到以 R15 值为地址的程序执行。								
	R12(IP)			R12					R12_fiq
	R13(SP)	R13	R13_svc	R13_abt	R13_und	R13_irq			R13_fiq
	R14(LR)	R14	R14_svc	R14_abt	R14_und	R14_irq			R14_fiq
	R15(PC)			R15					
状态寄存器	CPSR			CPSR					

程序状态寄存器 CPSR

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器							
		用户	系统	管理	中止	未定义	中断	快中断	
通用寄存器和程序计数器	R0(a1)								R0
	R1(a2)								R1
	R2(a3)								R2
	R3(a4)								R3
	R4(v1)								R4
	R5(v2)								R5
	R6(v3)								R6
	R7(v4)								R7
	R8(v5)								R8_fiq
	R9(SB,v6)								R9_fiq
	R10(SL,v7)								R10_fiq
	R11(FP,v8)								R11_fiq
	R12(IP)								R12_fiq
	R13(SP)	R13	R13_svc	R13_abt	R13_und	R13_irq			R13_fiq
	R14(LR)	R14	R14_svc	R14_abt	R14_und	R14_irq			R14_fiq
状态寄存器	CPSR			CPSR					

所有模式都可以访问当前程序状态寄存器 CPSR。每种异常都有自己的保存程序状态寄存器 SPSR。在进入异常时它保存 CPSR 的当前值，异常退出时可通过它恢复 CPSR。即
异常出现时 CPSR → SPSR
返回时 CPSR ← SPSR

2.6.2 Thumb 状态下的寄存器组织

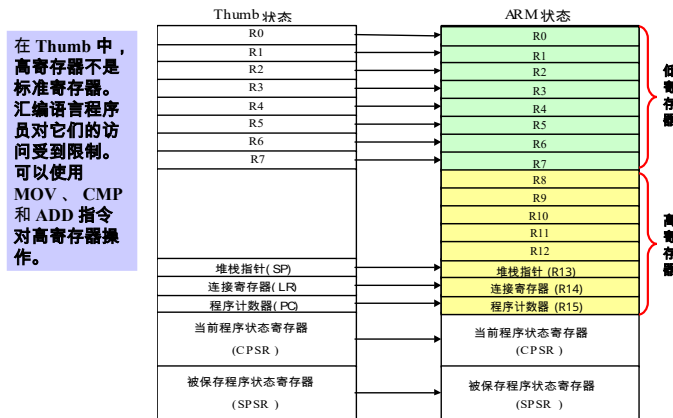
- 直接访问：8 个通用寄存器 (R7 ~ R0)、程序计数器 (PC)、堆栈指针 (SP)、连接寄存器 (LR) 和 CPSR。
- 每种特权模式下都有自己的 SP、LR 和 SPSR

THUMB State General Registers and Program Counter					
System & User	FIQ	Supervisor	Abort	IRQ	Undefined
R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7
SP	SP_fiq	SP_svc	SP_abt	SP_und	SP_fiq
LR	LR_fiq	LR_svc	LR_abt	LR_und	LR_fiq
PC	PC	PC	PC	PC	PC
THUMB State Program Status Registers					
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
	SPSR_fiq	SPSR_svc	SPSR_abt	SPSR_irq	SPSR_und

条件标志位

标志位	含义
N	N=1：运算结果为负数 N=0：运算的结果为正数或零
Z	Z=1：运算结果为零 Z=0：运算结果不为零
C	有 3 种情况会改变 C 的值： ◇ 加法运算（包括比较指令 CMN）：C=1：运算结果产生进位 C=0：没有进位 ◇ 减法运算（包括比较指令 CMP）：C=0：运算时产生借位 C=1：没有借位 ◇ 对于包含移位操作的非加/减运算指令，C 为移出值的最后一位
V	V=1：溢出（有符号数）
Q	在 ARM V5 及以上版本的 E 系列处理器中，Q=1：增强 DSP 运算溢出

Thumb 寄存器在 ARM 寄存器上的映射

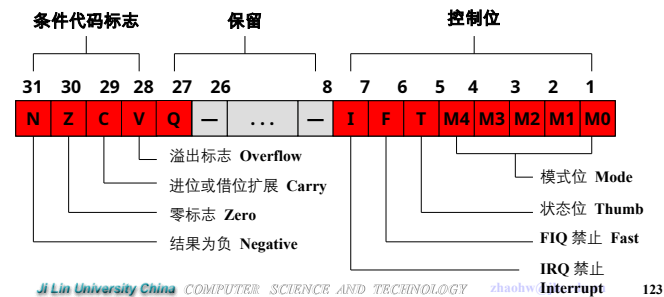


控制位

控制位	含义
I	I=1：禁止 IRQ 中断，I=0：允许 IRQ 中断
F	F=1：禁止 FIQ 中断，F=0：允许 FIQ 中断
T	对于 ARM V4 以上版本 T 系列，T=0：执行 ARM 指令；T=1：执行 Thumb 指令 对于 ARM V5 以上版本非 T 系列，T=0：执行 ARM 指令；T=1：强制下一条执行的指令产生未定义指令中断
M[4:0]	10000 用户模式 PC，R0-R14，CPSR
	10001 快速中断模式 PC，R0-R7，R8_fiq-R14_fiq，CPSR，SPSR_fiq
	10010 外部中断模式 PC，R0-R12，R13_irq-R14_irq，CPSR，SPSR_irq
	10011 管理模式 PC，R0-R12，R13_svc-R14_svc，CPSR，SPSR_svc
	10111 中止模式 PC，R0-R12，R13_abt-R14_abt，CPSR，SPSR_abt
	11011 未定义指令模式 PC，R0-R12，R13_und-R14_und，CPSR，SPSR_und
	11111 系统模式 PC，R0-R14，CPSR

2.6.3 程序状态寄存器

- CPSR：当前程序状态寄存器，可以在任何工作模式下被访问。
- SPSR：保持程序状态寄存器，只有在异常模式下，才能被访问；各异常模式拥有自己的 SPSR。发生异常时，SPSR 保存 CPSR 的值，格式同 CPSR。
- 状态标志：5 个，N 符号位，Z 零标志，C 进位，V 溢出位，Q DSP 运算溢出位。
- 控制标志：4 个，I 中断允许，F 快速中断允许，T 状态选择，M[4:0] 处理器工作模式



2.7 异常 (Exceptions)

- 异常：处理器由于外部或内部的原因，停止执行当前任务，转而处理特定的事件，处理完后返回原程序，继续执行。
- 例如处理一个外部的中断请求。在处理异常之前，当前处理器的状态必须保留，这样当异常处理完成之后，当前程序可以继续执行。
- 处理器允许多个异常同时发生，它们将会按固定的优先级进行处理。
- ARM 体系结构中的异常，与 8 位 /16 位体系结构的中断有很大的相似之处，但异常与中断的概念并不完全等同。

2.7.1 ARM 体系结构所支持的异常类型

- ARM 共有 7 种类型的异常，不同类型的异常将导致处理器进入不同的工作模式，并执行不同特定地址的指令。

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优 先 级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

指令预取中止

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优 先 级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

- 当出现下列情况之一，存储器会向处理器发出中止信号：
 - (1) 预取指令的地址不存在。
 - (2) 不允许当前指令访问。
- 预取指令被执行时才会产生该类异常。

2.7.2 异常优先级

- 多个异常同时发生时，ARM9 处理器将按照异常的优先级高低顺序处理。
- 异常优先级由高到低的排列次序如表所示 (1 最高)。

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优 先 级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

复位

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优先级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

当 nRESET 信号由低变为高电平时，ARM 处理器执行下列操作：

1. 强制 CPSR 中的 M[4:0] 变为 b10011 (管理模式) ；
2. 复位 CPSR 中的 I 和 F 位 ；
3. 清零 CPSR 中的 T 位 ；
4. 强制 PC 从地址 0x00 开始对下一条指令进行取指 ；
5. 返回到 ARM 状态并恢复执行 。

- 复位异常时，处理器立即停止当前程序，进入禁止中断的管理模式。
- 从地址 0x00000000 处开始执行。

数据访问中止

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优先级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

- 当出现下列情况之一，产生数据中止异常：
 - (1) 数据访问指令的地址不存在。
 - (2) 不允许当前指令访问时。

2.7.3 异常的响应及返回

- 异常发生会使得正常的程序流程被暂时停止，例如 ARM9 处理器响应 IRQ 异常。
- 处理器进入异常处理程序前，应该保存其当前的状态，以便当异常处理程序完成后，处理器能回到原来程序的断点处继续执行。

未定义指令、软件中断

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优先级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

- 当前指令未定义时，便产生未定义指令中断。
- 用户模式下使用指令 SWI 时，处理器便产生软件中断，进入管理模式，以调用特权操作。

外部中断请求、快速中断请求

异常类型	工作模式	特定地址 (低 端)	特定地址 (高 端)	优先级
复位	管理模式	0x00000000	0xFFFF0000	1
未定义指令	未定义指令中止模式	0x00000004	0xFFFF0004	6
软件中断 (SWI)	管理模式	0x00000008	0xFFFF0008	6
指令预取中止	中止模式	0x0000000C	0xFFFF000C	5
数据访问中止	中止模式	0x00000010	0xFFFF0010	2
外部中断请求 (IRQ)	外部中断模式	0x00000018	0xFFFF0018	4
快速中断请求 (FIQ)	快速中断模式	0x0000001C	0xFFFF001C	3

- 外部中断请求引脚有效，且 CPSR 中的 I=0 (允许) 时，产生 IRQ 异常。
- 快速中断请求引脚有效，且 CPSR 中的 F=0 (允许) 时，产生 FIQ 异常。

对异常的响应

- (1) 将下一条指令的地址存入相应连接寄存器 LR，以便程序在处理异常返回时能从正确的位置重新开始执行。
 - 1) 异常从 ARM 状态进入：LR 寄存器中保存的是下一条指令的地址 (一般是当前 PC -- 4，与异常的类型有关) ；
 - 2) 异常从 Thumb 状态进入：LR 寄存器中保存当前 PC 的偏移量，这样，异常处理程序就不需要确定异常是从何种状态进入的。
- (2) 将 CPSR 复制到相应的 SPSR 中。
- (3) 根据异常类型，强制设置 CPSR 的运行模式位。
- (4) 强制 PC 从相关的异常向量地址取一条指令执行，从而跳转到相应的异常处理程序处，同时设置中断禁止位，以禁止中断发生。
- 注意：如果异常发生时，处理器处于 Thumb 状态，则当异常向量地址加载入 PC 时，处理器自动切换到 ARM 状态。

异常返回

- 异常处理完毕之后，ARM 微处理器会执行以下几步操作从异常返回：

- (1) 将链接寄存器 LR 的值减去相应的偏移量后送到 PC 中。
- (2) 将 SPSR 复制回 CPSR 中。
- (3) 若在进入异常处理时设置了中断禁止位，要在此清除。

- 注意：**恢复 CPSR 的动作会将 T、F 和 I 位自动恢复为异常发生前的值。

SWI 和未定义指令时的 LR

- SWI 和未定义指令异常中断是由当前执行的指令自身产生的。
- 当 SWI 和未定义指令异常中断产生时，PC 的值还未更新，它指向当前执行指令后面的第 2 条指令。
- 当 SWI 和未定义指令异常中断发生时，ARM 将值 (PC-4) 保存到异常模式下的 LR (R14_svc 或 R14_und) 中。这时 (PC-4) 即指向当前指令的下一条指令。
- 返回操作：**可通过指令“MOVS PC, LR”来实现。同时，SPSR_svc 或 SPSR_und 的内容被复制到当前程序状态寄存器中。



IRQ 与 FIQ 时的 LR

- ARM 执行完当前指令后，查询 IRQ 与 FIQ，若有中断请求，且 CPU 允许中断，将产生 IRQ 中断或 FIQ 中断。
- 当 IRQ 或 FIQ 中断产生时，PC 的值已经更新，它指向当前执行指令后面的第 3 条指令。IRQ 或 FIQ 中断发生时，ARM 将值 (PC-4) 保存到异常模式下的 LR (R14_irq 或 R14_fiq) 中。这时 (PC-4) 即指向当前指令后的第 2 条指令。
- 返回操作：**可通过指令“SUBS PC, LR, #4”来实现。同时，SPSR_irq 或 SPSR_fiq 的内容被复制到当前程序状态寄存器中。



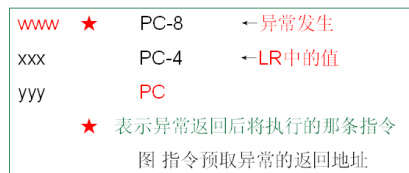
异常处理时保存在 R14 中值

- 下表总结了进入异常处理时保存在相应 R14 中的 PC 值，及在退出异常处理时推荐使用的指令。
- 表中 * = 当前执行指令地址。

异常及指令	返回指令	进入异常处理时保存在 R14 中值	
		ARM R14_x	Thumb R14_x
BL 指令	MOV PC, R14	* + 4	* + 2
SWI	MOVS PC, R14_svc	* + 4	* + 2
未定义指令	MOVS PC, R14_und	* + 4	* + 2
FIQ	SUBS PC, R14_fiq, #4	* + 4	* + 4
IRQ	SUBS PC, R14_irq, #4	* + 4	* + 4
预取中止	SUBS PC, R14_abt, #4	* + 4	* + 4
数据中止	SUBS PC, R14_abt, #8	* + 8	* + 8

指令预取中止时的 LR

- 指令预取中止异常中断是由当前执行的指令自身产生的。发生异常后，程序要返回到这个有问题的指令处，重新读取并执行该指令。
- 指令预取中止异常中断产生时，PC 的值还未更新，它指向当前执行指令后面的第 2 条指令。此时，ARM 将值 (PC-4) 保存到异常模式下的 LR (R14_abt) 中。这时 (PC-4) 即指向当前指令的下一条指令。
- 返回操作：**可通过指令“SUBS PC, LR, #4”来实现。同时，SPSR_abt 的内容被复制到当前程序状态寄存器中。

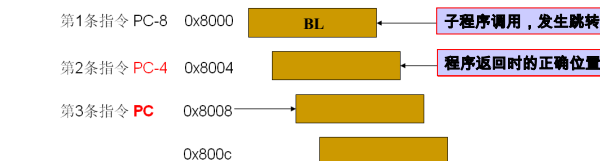


2.7.4 应用程序中的异常处理

- 当系统运行时，异常可能会随时发生。
- 为保证在 ARM 处理器发生异常时不至于处于未知状态，在应用程序的设计中，首先要进行异常处理，采用的方式是在异常向量表中的特定位置放置一条跳转指令，跳转到异常处理程序。
- 当 ARM 处理器发生异常时，程序计数器 PC 会被强制设置为对应的异常向量，从而跳转到异常处理程序，当异常处理完成以后，返回到主程序继续执行。

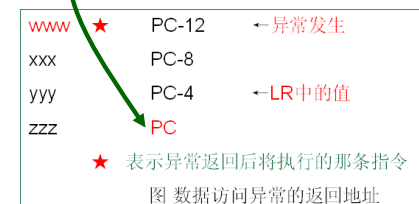
子程序调用时的 LR

- 执行第 1 条指令 (地址 0x8000) 时，PC 正指向第 3 条指令 (地址 0x8008)。第 1 条指令是调用指令 BL。
- 执行完子程序返回时，PC 应指向第 2 条指令 (地址 0x8004)。
- 第 1 条指令执行时，ARM 硬件自动把 PC (=0x8008) 保存至 LR，并将 LR 自动调整为：LR=LR-0x4 (即 LR=0x8004)。
- 最终保存在 LR 中的是第 2 条指令的地址，正好是正确的返回地址。
- 由于各种异常中断响应的过程不同，因此，保存在 LR 中的地址是不同的。大多数情况下，保存在 LR 中的地址是：LR=PC-4。



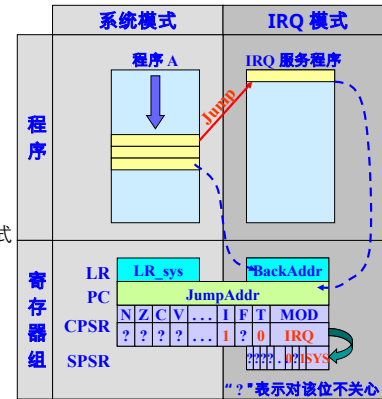
数据访问中止时的 LR

- 数据访问中止异常中断是由数据访问指令自身产生的。发生异常后，程序要返回到这个有问题的数据访问处，重新访问该数据。
- 数据访问中止异常中断产生时，PC 的值已经更新，它指向当前执行指令后面的第 3 条指令。此时，ARM 将值 (PC-4) 保存到异常模式下的 LR (R14_abt) 中。这时 (PC-4) 即指向当前指令后的第 2 条指令。
- 返回操作：**可通过指令“SUBS PC, LR, #8”来实现。同时，SPSR_abt 的内容被复制到当前程序状态寄存器中。



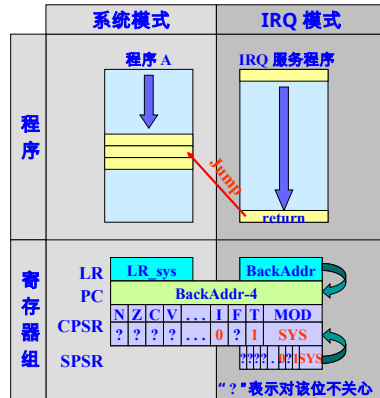
进入异常过程

- 用户程序运行时发生 IRQ 中断，硬件完成以下动作：运行用户程序，假定当前处理器状态为 Thumb 状态，允许 IRQ 中断；将 CPSR 寄存器内容存入 IRQ 模式的 SPSR 寄存器；置位 I 位 (禁止 IRQ 中断)；清零 T 位 (进入 ARM 状态)；设置 MOD 位，切换处理器模式至 IRQ 模式；将下一条指令的地址存入 IRQ 模式的 LR 寄存器；将跳转地址存入 PC，实现跳转。



在异常处理结束后，异常处理程序完成以下动作：

- 将 SPSR 寄存器的值复制回 CPSR 寄存器；
- 将 LR 寄存器的值减去一个常量后复制到 PC 寄存器，跳转到被中断的用户程序。



bootloader 源代码外部中断处理实例 .3

(3) 中断向量地址分配 (内存表数据结构说明)。

MAP_ISR_STARTADDRESS	#	4	; 为中断向量分配地址空间
SYS_RST_VECTOR	#	4	; 此空间为 RAM 的地址
UDF_INS_VECTOR	#	4	
SWI_SVC_VECTOR	#	4	
INS_ABT_VECTOR	#	4	
DAT_ABT_VECTOR	#	4	
RESERVED_VECTOR	#	4	
IRQ_SVC_VECTOR	#	4	
FIQ_SVC_VECTOR	#	4	

中断向量设置程序：

```
ldr r0, IRQ_SVC_VECTOR
ldr r1, =IsrIRQ ;IsrIRQ 为中断服务程序的入口地址
str r1, [r0] ;将其写入中断向量 IRQ_SVC_VECTOR
中
```

bootload 代码固化在 flash 中，只读

(1) 整个地址空间的起始位置 (地址 0x00000000 开始) 有以下指令。

- 当发生外部中断请求时：
 - 处理器自动保存当前状态 (PC→R14 , CPSR→SPSR)
 - 进入外部中断模式
 - 执行地址 0x00000018 处的指令，跳转到 IRQ_SVC_HANDLER 执行。

b	SYS_RST_HANDLER	;0x00000000
b	UDF_INS_HANDLER	;0x00000004
b	SWI_SVC_HANDLER	;0x00000008
b	INS_ABT_HANDLER	;0x0000000c
b	DAT_ABT_HANDLER	;0x00000010
b		
b	IRQ_SVC_HANDLER	;0x00000018
b	FIQ_SVC_HANDLER	;0x0000001c

ROM 空间

! 注意存放的存储空间

(2) IRQ_SVC_HANDLER 处的代码如下。

```
IRQ_SVC_HANDLER
sub lr, lr, #4
stmfd sp!, {r0-r3, lr}
ldr r0, =IRQ_SVC_Vector
ldr pc, [r0]
```

修正返回地址，IRQ 模式减一个固定偏移量。

将通用寄存器和返回地址压入堆栈

IRQ_SVC_Vector 为外部中断请求的中断向量，是一个 RAM 空间的地址，将该地址本身送入寄存器 r0

r0 间接寻址，将其所指的 RAM 字单元的内容取到 PC 中，该内容是 IRQ 的真正的中断服务程序入口地址

第2章 结束